
The Educado Project - Web Application

- A digital learning platform developed to educate waste pickers in Brazil -

Project Report
Group 6 - Virtual Tutor group

Copyright © Aalborg University 2023

Report is written in LaTeX Overleaf. Zotero integration with Overleaf is used for the sources and citations in the report. Program is written in Visual Studio Code with the programming language TypeScript JavaScript, and shared using GitHub. Google Docs is used for sharing notes. Diagrams are either found from other sources and cited. PBI listing and work distribution have been made through the web application Miro allowing everyone to share their development. The Figma web application was used as a style guide which was developed by the design team in Brasília.



Software
Aalborg University
<http://www.aau.dk>

AALBORG UNIVERSITY

STUDENT REPORT

Title:

Educado - Virtual Tutor

Theme:

Software

Project Period:

Fall Semester 2023

Project Group:

6

Participant(s):

Goran Kirovski
Karina Mary-Anne Botes
Magne Frank Dinesen
Pernille Knudsen
Tobias Frieze
Viktor Johannes Hansen

Supervisor(s):

Daniel Russo

Abstract:

This report consists of an Agile Development cycle regarding the development of the Web and App project Educado. In this report, you will be presented with this group's experience, learnings and considerations through the whole collaborative development. The group will also present different theoretical aspects and thoughts, considerations and decisions in what theoretical methods and physical methods were made during the work process. This project will also give the reader an insight into how the group tackled their first collaborative programming project which involved unique, difficult and cooperative conflicts. This will also give insight into how the group managed to work under management from a product manager, and how the group adapted to a real-life target audience with availability for engagement in the form of user testing and other forms of feedback. What the group learned and what the group ended up achieving throughout this project is reflected in the discussion and conclusive part of the ending of the report.

Copies: 1

Page Numbers: 75

Date of Completion:

December 20, 2023

Contents

1	Introduction	8
1.1	AI Assistance	8
1.2	Scope of the project	8
1.3	Background for the project	8
1.4	About Educado	10
2	Initial State	11
2.1	The overall structure and architecture of the initial code base	11
2.1.1	Inconsistent structure	11
2.1.2	Initial state of the back end	12
2.1.3	Initial state of the web front end	12
2.1.4	Initial state of mobile	12
2.1.5	Summary	12
2.2	Database diagram	13
3	Important Concepts	13
3.1	Scrum roles	13
3.2	Events in scrum	14
3.2.1	The Sprint	14
3.2.2	Sprint Planning	14
3.2.3	Daily Scrum	15
3.2.4	Sprint reviews	15
3.2.5	Sprint Retrospective	15
3.3	General Concepts	15
3.3.1	Product Backlog	16
3.3.2	Sprint Backlog	16
3.3.3	Increment	16
4	Project Structure	16
4.1	Nexus Scrum Framework	16
4.1.1	Role and Impact of the integration team	17
4.2	Development process	17
5	Clean Architecture	20

6	Microservice	21
6.1	Technology Stack and Tools	21
6.2	Communication standards	21
7	Sytem Architecture	21
7.1	The Educado platform architecture	21
7.1.1	Educado Certificate Service	22
7.1.2	Educado Video Service	23
7.2	Architecture reflections	24
8	Data Models	24
9	Sprints	26
9.1	Preparation prior to the sprints	26
9.2	Sprint 1	26
9.2.1	Sprint 1 planning	26
9.2.2	Daily scrum meetings	27
9.2.3	Integration team	27
9.2.4	Increments	27
9.2.5	Sprint 1 Review	28
9.2.6	Sprint 1 Retrospective	28
9.3	Sprint 2	29
9.3.1	Sprint 2 planning	29
9.3.2	Daily scrum meetings	29
9.3.3	Integration team	29
9.3.4	Jest testing	30
9.3.5	Increments	30
9.3.6	Sprint 2 Review	31
9.3.7	Sprint 2 Retrospective	31
9.4	Sprint 3	32
9.4.1	Sprint 3 planning	32
9.4.2	Daily scrum meetings	32
9.4.3	Integration team	32
9.4.4	Increments	33

9.4.5	Sprint 3 Review	34
9.4.6	Sprint 3 Retrospective	35
9.5	Sprint 4	35
9.5.1	Sprint 4 planning	35
9.5.2	Daily scrum meetings	36
9.5.3	Integration team	36
9.5.4	Increments	36
9.5.5	User test	37
9.5.6	Sprint 4 Review	38
9.5.7	Sprint 4 Retrospective	39
9.6	Sprint 5	39
9.6.1	Sprint 5 planning	39
9.6.2	Cypress	41
9.6.3	Daily scrum meetings	41
9.6.4	Integration team	42
9.6.5	Increments	42
9.6.6	Sprint 5 Review	42
9.6.7	Sprint 5 Retrospective	43
9.7	Sprint 6	43
9.7.1	Sprint 6 planning	43
9.7.2	Daily scrum meetings	44
9.7.3	Integration team	44
9.7.4	Increments	44
9.7.5	Sprint 6 Review	47
9.7.6	Sprint 6 Retrospective	47
10	Implementation	48
10.1	Front End	48
10.1.1	Submitting sections	48
10.1.2	Submitting the list of components	49
10.1.3	Course creation	50
10.1.4	Tooltip	52

10.2 Backend	54
10.2.1 Get course from id	54
10.2.2 Creating a new lecture	55
10.2.3 Updating a section components	56
10.2.4 Deleting a section	57
10.3 Test	58
10.3.1 Testing through Cypress	58
10.3.2 Backend testing	61
11 Final Retrospective	65
11.1 Sprint reviews and sprint retrospectives	65
11.1.1 Sprint 1	65
11.1.2 Sprint 2	65
11.1.3 Sprint 3	66
11.1.4 Sprint 4	66
11.1.5 Sprint 5	66
11.1.6 Sprint 6	67
11.2 Progress	68
11.3 Our Use Of The Nexus Scaling Framework	68
11.3.1 Nexus within the project	68
11.3.2 Evaluation	69
11.4 Communication and challenges	69
11.4.1 Task Board	69
11.4.2 Integration team issues	69
11.5 Product owner	70
12 Discussion	71
12.1 Nexus framework	71
12.2 Management of the groups	71
12.3 Management of the project	71
12.4 Technical debt	72
12.5 Testing with Cypress vs other testing frameworks	73
12.6 Future development	73

13 Conclusion	75
Appendix A - Checkbox Implementation	79

1 Introduction

1.1 AI Assistance

Throughout the process of making this project, the group utilized artificial intelligence (AI) assistance to help develop some code and comments for the project. ChatGPT and GitHub Co-pilot have been beneficial in helping to suggest solutions to bugs, generate ideas, generate relevant comments to function, and more. The group was also incentivized by the supervisor to utilize AI, hoping that it would enhance the overall quality and structure of the code.

The main goal of the implementation of AI assistance was to help with increasing the overall quality of life. Helping with subjects like unit testing and generating fields. Which, in general, saves only a few seconds but adds up to a larger amount of time saved overall.

Additionally, ChatGPT was utilized to brainstorm ideas for sentences or to assist with refining and restructuring troublesome sentences, contributing to improving the clarity and expression in the project documentation.

The goal was also to use the potential of AI to further our understanding and prowess in the use of AI, hoping it would result in the experimental use of AI forming a greater potential in the context of creating better and more sustainable code.

1.2 Scope of the project

This report is a part of the 5th-semester project in Software Engineering at Aalborg University in Copenhagen. This semester, each student can individually choose between complex backend or complex frontend software and work together as a full-stack development team. The aim of the project is - as stated on Aalborg University's (AAU) official website - *"to obtain knowledge and skills in analysis, design, implementation and assessment of complex software systems in a larger development environment"* [1].

In this semester's project, multiple teams work together on the same project. In total, there are six full-stack teams, and each team is made up of between five and six students. Each team has its theme for the project and its product backlog items (PBIs) that they are responsible for, but the final product will be the same for all groups. Though all the teams are collaborating on the same project, some parts of the project are more relevant for some teams than others. The final product will be represented as both a website and an Android application. Therefore, some teams mainly focus on the website, while others focus on the Android application.

To coordinate and work with multiple teams on the same project, finding a good development methodology is essential. The Agile software development methodology, with the use of the Scrum framework scaled with Nexus, has been used to ensure high-quality software development.

In addition to working in a larger development environment, an important part of this project is to inherit code written by other students and use this for further development. The code written in this project is based on a four-year-old code base from the students of AAU in Aalborg in 2019. Last year, in 2022, it was rewritten and further developed by 5th-semester software students in Copenhagen. Even though the initial sentiment was to avoid modifying the existing code base where possible, a lot of changes and reworks of the inherited code were needed. This year UI designers, who are students from the University of Brasília, created a completely new design. In addition, the code had not been maintained for over a year and did not follow the guidelines for this year's project.

1.3 Background for the project

Brazil is the world's fifth largest country and has over 215 million citizens [2]. Large economic inequalities and corruption have characterized the country's recent politics. Numbers from the World Bank show that even though the country has experienced some economic growth in the last couple of years, poverty is still a huge problem [3]. In a report made by the Brazilian Institute of Geography and Statistics (IBGE), called *"Síntese de indicadores sociais - uma análise das condições de vida"*, (English translation: *Synthesis of Social Indicators - An Analysis of Living Condition*), the living conditions of the Brazilian citizens were studied. They found out that in 2021, according to criteria of the World Bank, 62.5 million people, which is 29.4% of the population in Brazil, were living in poverty. Among those, 17.9 million were in extreme poverty, which is 8.4% of the population [4], see figure 1.

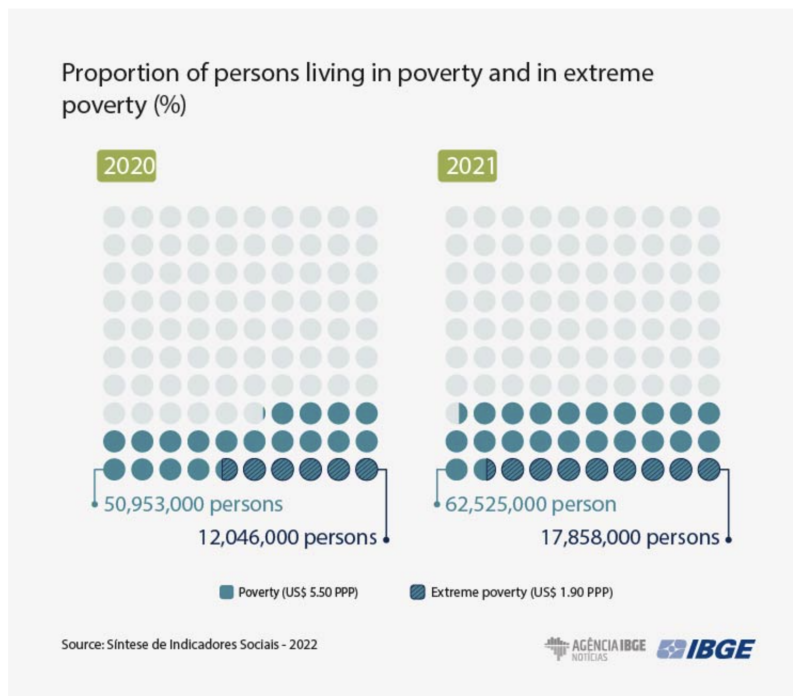


Figure 1: Figure showing the proportion of persons living in poverty in Brazil [4].

The data also shows that for people whose salaries are up to 1/4 of minimum wage, their salary only represented 53.8% of their total income, whereas 34.7% comes from social programs. André Simões, the analyst for the report, stated that *“Those are more vulnerable housing units with limited access to the labor market. Therefore, social programs end up having a bigger impact”* [4].

Poverty in Brazil is a huge and complex problem and many factors explain why. However, the lack of quality education is a part of it.

When it comes to education, data from 2022 shows that:

- 5,6% of the population aged 15 (~9,6 mil. people) is illiterate [5]
- 55% of those people are in the northeast region of the country [5]
- 3% of white people aged 15 are illiterate, while 8% of black or brown people aged 15 are illiterate [5]
- 53% of the population over 25 completed high school [5]
- 19% of the population has a higher education degree [5]

Another important factor is the lack of jobs. 40% of young people aged between 22 and 25 with higher education do not have a qualified job [6]. In addition, 8% of working-aged people are not working but are available and trying to find work. This means that 8.6 million people are unemployed [7]. As a consequence of the lack of jobs, many people work in low-paid and lower-value-added jobs.

Waste pickers are one of these vulnerable groups working in low-paid jobs. Numbers from 2021 show that there were around 800 thousand waste pickers in Brazil and 60% of them were women [8]. In 2018, Brazil achieved a significant milestone by closing one of the world’s major landfills. To solve this issue, the government established sorting facilities, allowing waste pickers to maintain their income stream while continuing to contribute to the city’s recycling infrastructure. This action not only improved the working conditions for waste pickers by reducing their exposure to health risks but also contributed to mitigating environmental consequences [9].

However, this initiative had economic implications for the waste pickers. Their transition from dumpsite work to employment in the newly established recycling centers worsened their economic situation. In particular, their

salaries did not adequately meet the rising cost of living, posing financial challenges for waste pickers in adapting to their new roles. Regardless of their low income, how they manage their finances can influence their economy. Waste pickers in Brazil lack basic knowledge about financial health, posing a challenge in managing their money efficiently [8].

A study conducted in 2016 found that 17.4% were illiterate, and 65% did not complete their education [9]. In terms of providing educational support, stabilizing their current situation, and managing their economy, these are vital aspects that require attention [9]. In a focus group with Brazilian waste-pickers, it was found that they feel "wasting time on education means wasting the time of collection and, this way, wasting money" [8].

1.4 About Educado

The project is part of the Mobile Education Project (MEP), which was formed in collaboration with Aalborg University (AAU) and the University of Brasilia (UNB) in 2019 for the SGDC initiative. Educado is a part of the MEP with a focus on improving the life situation of waste pickers in Brazil. At UNB, the students of this semester and last semester have been working on user journey mapping, user interface design, content creation, and product management in terms of users' needs and journeys. The project is done in close collaboration with the product owner, the stakeholders, and the intended users of the product in Brazil. AAU will be responsible for the development of the Educado mobile application and the web application, while UNB will be responsible for the design and requirement criteria.

As part of the Educado project, this collaborative effort was dedicated to enhancing the educational prospects of waste pickers through digital learning. Educado serves as a platform designed to connect with waste pickers and address their educational needs. The primary objective is to offer free access to educational resources and improve the quality of life for the Brazilian population. To maintain user engagement, the app will also offer education through a gamified experience. The layout will be created to provide users with visual designs and animations that will reward them for learning and provide the maximum amount of educational content possible. For the content to be accessible to the target group, there will be a specific emphasis on the development of a web application for qualified content creators. The content creators should, in the web application, be able to provide a variety of courses, which should be appealing to the users.

The project also takes part in both the Sustainable Development Goals (SDG) and the European Region Action Scheme for the Mobility of University Students (ERASMUS). ERASMUS is the European Union's program to support education, training, youth, and sport in Europe [10]. Educado was one of the projects that received funding this year (2023). The SDG is the United Nations' 17 goals for sustainable development [11]. A meaningful part of the work done by software developers and engineers is developing solutions that contribute to a better future. A way of making sure of that is to have the SDGs in mind. In Brazil, waste pickers and other low-income groups lack access to basic and professional education and have poor social conditions. Therefore, as stated by the projects' product owner (PO), Luiza Cardoso Queiroz Melo, this project aims to provide access to a tailored educational experience in an easy, quick, and dynamic way, to keep the waste pickers engaged and interested in learning [8]. The project's main focus will be to achieve this by providing a mobile education platform for waste pickers in Brazil. However, the project can be up-scaled and help not only waste pickers but also other vulnerable groups around the world. Therefore, if the project is successfully implemented, one could argue that it will affect multiple SDGs. Such as goal number four: *quality education* and goal number eight: *decent work and economic growth* [11].

When it comes to the education part, there are multiple ways the Educado project can have an effect. On the one hand, one can use the Educado app as inspiration and a stepping stone into applying to different educations. On the other hand - if there are quality professors that contribute with different courses on the app, and if Educado could collaborate with universities as well as both governmental and private organizations, the app could create certificates that would be approved as a form of alternative education. This way, Educado can work as an alternative education itself. Either way, it could be argued that all forms of education are great, and especially for vulnerable people, even a basic education can have a significant positive effect on their quality of life. HERE THE COLLABORATIVE PART ENDS

2 Initial State

2.1 The overall structure and architecture of the initial code base

The project that was handed over consisted of three main repositories; 'Educado-backend', 'Educado-frontend', and 'Educado-mobile'. The front end repository had the code for the web application, while the mobile repository contained the mobile front end. The back end was the central point of access for both of the front ends to access and manipulate users and the state of their resources. The mobile and back end repositories were written in JavaScript and the web front end repository was in TypeScript. The platform for the mobile application development was Expo, and it used react-native as its framework. The web application used Vite as its development environment and React as its framework.

The structure of the project is that all the business logic for both the web application and the mobile application can be found in the back end repository. The back end runs independently of the front end repositories and is accessed via its API. This contributes to better separation of concerns. In addition, all repositories have a layered structure, following some clean architecture principles, as well as some Model-View-Controller (MVC) principles. Since the back end is deployed independently and is implemented into the UI through its API, it could be argued that it is not a pure monolithic structure. However, one may argue that it cannot be defined as pure Service-Oriented Architecture (SOA) either. To follow SOA the back end should be separated into smaller microservices. One common way of implementing SOA is by using microservices. This code base is inherited and therefore, the reflection and intention behind the structure is unknown. Looking at the initial code base, one can assume that the structure of the project somehow emerged by accident, and as such seemingly lacks an overarching design. This is also mirrored in the file structure, as the various repositories felt messy and disorganized in places. As a consequence, the code base was, unintentionally, lacking modularity and flexibility.

2.1.1 Inconsistent structure

The Educado project that was handed down from last year's students was lacking in its architecture. Even though the back end was developed independently from the front end, it was not divided into smaller services. If the project used microservices, it would improve the project's modularity and integrity. When using microservices, functionalities are distributed into smaller microservices. The different microservices can be written in different languages and they all serve different purposes. This way, one can use the programming language most suitable for the task that the software aims to solve. This might also improve its overall quality because specific frameworks might cater to specific needs and characteristics. It also helps with downtime since updates can be made to specific services without having to the whole platform.

The fact that no microservices were used initially indicates that the project worked on was still in very early stages of development. It may also indicate that the project's architecture was not well planned from the beginning and may have arisen spontaneously. Not fully implementing SOA and microservices may ease the development process in the beginning. However, when up-scaling, this will cause problems. Therefore, one could assume that the choice of not separating the back end into multiple microservices was made in the early stages of the project. An important factor to remember when talking about the structure is that the initial code base is more than two years old. It may be that last year's students inherited a more monolithic architecture, trying to refactor into a more SOA, but did not manage to fully refactor it in time.

Throughout the repositories, the impression was that there inherently was a messy structure along with unrefined code and files without any particular purpose. One key reason for this is the lack of comments in the code. When inheriting code, providing relevant comments and/or documentation is essential. In addition to using well-defined and descriptive file names, folder names, function names, and variable names. This was heavily lacking in the code base and resulted in unclear functions that seemed to serve no purpose. Furthermore, this made it difficult to fix any bugs that might occur, as the knowledge about modules and how they were coupled had gone with the previous developers. It resulted in a steeper learning curve for everyone, which also meant that the maintainability of the project would be challenging in the future. As a consequence, it was invested crucial time to establish a clear and organized code base to fix these issues.

2.1.2 Initial state of the back end

The project's back end carried issues such as unused routes, which made it difficult to navigate. This also created a lot of issues with figuring out what code did what, as quite a few routes carried very similar logic and returns. Cleaning the code required consistent communication between teams in the different repositories, which unfortunately were lacking at the start. This created issues with deleting routes where a few of them were still in use by some teams, like mobile and web needing to use the same data but using two different routes that made the structure of the data incompatible with each other. It was also clear that no API guidelines were used when creating the routes, which created a lot of confusion that resulted in double-checking to see if it was the correct route.

2.1.3 Initial state of the web front end

While the front end could be executed and was somewhat functional, it suffered a lot from primitive code. The login system was described as such, acting as a hurdle rather than a seamless part of the user experience. The course creation and editing were fully dependent on the data in the database, which made everything crash with the complete wipe of the database before the semester started.

Tailwind was used to style the pages, making the code more readable, as the styling is kept together with the page elements themselves. Using Tailwind helped with keeping a more uniform design palette. This ensured that all developers had access to the same resources, as well as it kept styling choices consistent throughout the entire front end. For some developers, Tailwind also serves as a very intuitive tool, because styling is written exactly where you need it, where you can pick and choose a lot more than e.g. vanilla CSS, where predefined CSS classes are attached to a piece of HTML to style it.

2.1.4 Initial state of mobile

The initial code base for the mobile application was impossible to run without errors at the beginning of the project. The reason behind this is a mix between deprecated libraries, as well as having an empty database leading to error displaying data. There were also certain files, notably `storageService.js`, that exhibited poor code quality scores according to Codescene. This software highlights areas that demand attention and refactoring. Moreover, the development of the mobile application lacked a clear visual design direction, leaving room for improvement in terms of user interface and user experience. The mobile front end was written using React Native, which focuses on writing modular and reusable components as in React for the web application. For styling the pages, NativeWind, and Tailwind for React Native, were used. As with Tailwind, NativeWind allows for defining shared styling like colors and fonts in one place as well as keeping the styling with their elements. These strengths, however, were not used to their full potential due to a lack of consistent implementation across the various components of the application. The sporadic application of these styling frameworks led to a fragmented user interface, with different screens and elements displaying inconsistent styles and themes.

Furthermore, since last year the design requirements for the mobile app had completely changed, which meant that most of the existing styling on the app already had to be completely changed. This meant of course that time at the beginning of this semester had to be allotted to this.

2.1.5 Summary

The overall architecture and structure of the code base require a strategic overhaul, emphasizing modularity, cleanliness, and adherence to best practices. By addressing these issues, the development teams can contribute to a more scalable, maintainable, and user-friendly application.

2.2 Database diagram

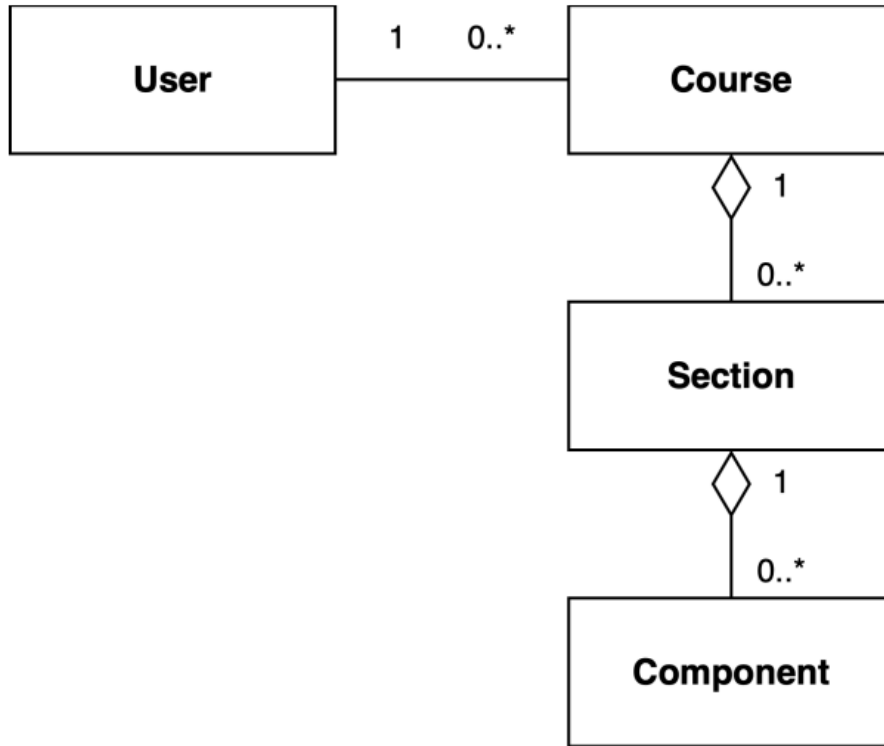


Figure 2: Figure of DB structure taken from one of the previous semester's reports [12].

The Database setup handed over had a rather simple setup. It consisted of 4 documents as seen in diagram 2. Users could have references to courses, courses could have references to sections, and so on. The references were only one way, for example, a section item would not have any information about which course item it belonged to.

3 Important Concepts

3.1 Scrum roles

As a part of the agile and scrum methods in software development, some roles and their responsibilities have to be defined and understood before the project occurs. The Scrum team primarily consists of three different roles, each having its responsibility in the project. These roles are product owner, Scrum master, and development team [13].

The Product Owner (PO) is responsible for setting a clear direction for the rest of the teams [14]. This actor is responsible for understanding the customer and business requirements and using that knowledge to communicate the backlog priorities and stakeholder wishes to the development team [14]. The product owner is responsible for managing and establishing a product backlog [15]. The product owner is also one of the key communicators between the development team and the stakeholders. All forms of refinement and reflection of the customer's voice are also factors the product owner manages [16].

The Scrum Master is in charge of planning, motivating, organizing, and coordinating the developers [13]. The scrum master is responsible for the facilitation of a smooth workflow, which entails hosting the daily scrum meetings, removing obstacles and internal conflicts. Additionally the Scrum master is responsible for improving the developers' Scrum framework practices and enticing the developers to work within the framework [15]. Furthermore, the scrum master assists in interpreting the PBIs, ensuring that the scrum team understands them correctly [17].

The Development team is a crucial component responsible for delivering a releasable increment at the end of each Sprint that meets the definition of done [18]. While the ideal size varies, it should be small enough for agility yet large enough for significant work completion [18]. The scrum trainer and CEO of the Scrum Academy GmbH Sohrab Salimi, recommends groups of three to nine people [19]. The developers' ability to form increments and adapt their plans daily to reach a sprint goal is crucial for effective teamwork [20]. Characteristics include self-organization, cross-functionality, no individual titles, and collective accountability [18]. Responsibilities involve sprint execution, daily scrum participation, refining the product backlog, planning the sprint, as well as inspecting and adapting both the product and process [18]. The team collectively strives for efficiency and continuous improvement through activities such as sprint reviews and retrospectives [18].

3.2 Events in scrum

This section will detail all the events relevant to the Scrum framework, as described in the Scrum Guide [21] and seen in Figure 3, as well as how these have been applied to the Educado project.

SCRUM FRAMEWORK

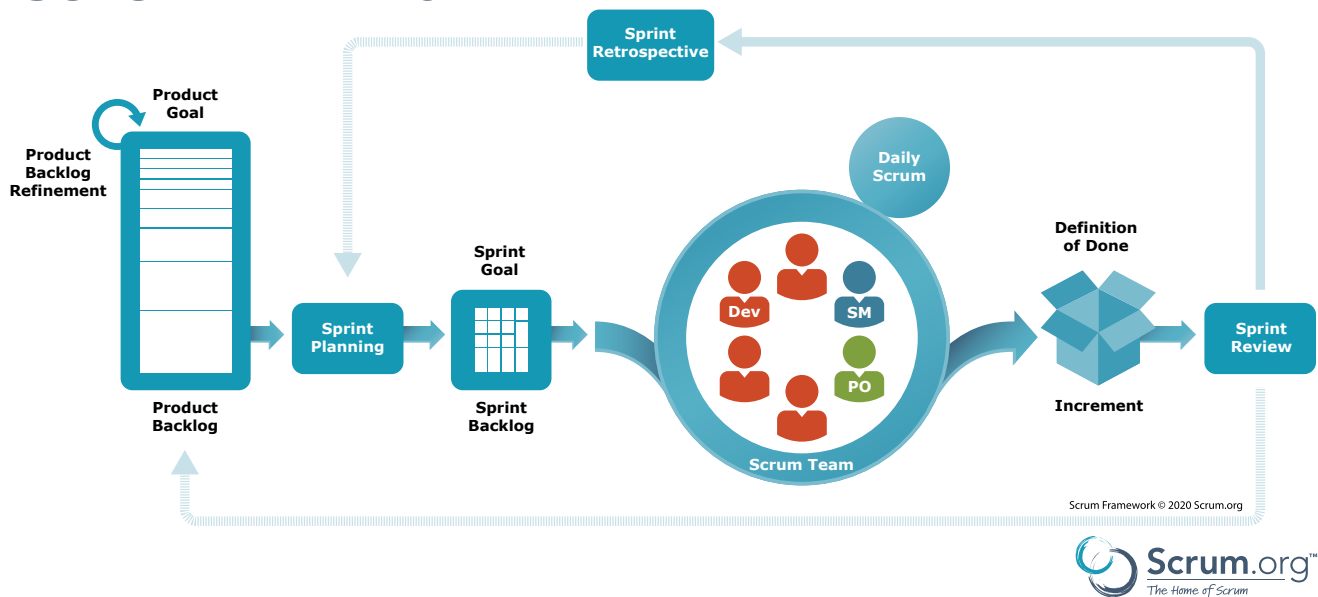


Figure 3: The scrum framework with the different events and how they tie together[22].

3.2.1 The Sprint

Sprints are the heartbeat of Scrum and are essentially the time allocated to working towards the current sprint goal. They have a fixed length of time, varying in length as decided by each organization, often in the time frame of one week to one month. Each sprint has a different sprint goal, as decided by the product owner, and all the work needed to complete the sprint goal is done in the relevant Sprint [21]. Whether or not the sprint goal is completed is the deciding factor for success in the Scrum framework. Anything that would endanger its completion has to be addressed properly. This could be changes to the product backlog or adjusting the scope of the sprint goal, as agreed upon by the product owner. It is important to keep in mind that employing longer sprints runs the risk of rendering the sprint goal obsolete; Scrum is mostly used to solve complex problems, in which consistent predictability is almost non-existent. A lot can change during a project, and having long sprints runs the risk of *too* much changing, making the sprint goal less relevant. Having a shorter sprint duration remedies this somewhat, as the sprint goals will be of lesser scope, making inevitable changes less impactful, and more manageable [21].

3.2.2 Sprint Planning

Sprint planning is the process of laying out the work that needs to be completed, to achieve the newly created sprint goal. One of the product owner's tasks in this event is making sure the Scrum team understands the importance of this sprint goal. The product owner and the individual Scrum teams will then discuss what can be done for this

sprint, by way of deciding what items from the product backlog they can and should be working on, concerning the sprint goal. The developers' job during sprint planning is to estimate the workload of the chosen tasks from the backlog. Then the product owner's job is to value each task to force a sense of priority upon the tasks. The team of developers should decide upon how the individual items will be implemented, turning items into increments is exclusively their responsibility [21].

3.2.3 Daily Scrum

A team of developers in a Scrum framework will start their work day with a meeting no longer than 15 minutes, in which they will inspect the progress they have made towards the current sprint goal. This is often referred to as 'Daily Scrum', and can be a very important tool for the individual development teams. Daily Scrum aids in improving communication between team members, ensuring that all the work that is done is relevant and meaningful while also identifying potential problems within the various aspects of the work [21].

3.2.4 Sprint reviews

The sprint review is dedicated to inspecting the work of each team in the sprint and presenting the increments to the key stakeholders. During the sprint review, the teams and stakeholders also review what has changed in the environment. They use this information to adjust plans and the backlog [21]. Key aspects of a sprint review include:

Demo of Completed Work: The development team demonstrates the functionality they have implemented during the sprint. This can include new features, enhancements, or bug fixes [23], [24].

Stakeholder Feedback: Stakeholders, including product owners, customers, and other relevant parties, are invited to attend the sprint review. They have the opportunity to provide feedback on the demonstrated work [24].

Collaborative Discussion: The sprint review is not just a presentation; it's a collaborative discussion. The team and stakeholders discuss what was done, what challenges were faced, and how they can improve in the future [23].

Review of the Product Backlog: The product backlog is reviewed by the product owner and stakeholders to ensure it is up to date. New items, changes in priority, and any adjustments to the backlog are discussed based on the feedback received during the sprint review [24].

Overall, the sprint review promotes transparency, collaboration, and continuous improvement within the Agile development process. It helps ensure that the product being developed aligns with the stakeholders' expectations and provides a mechanism for adapting to changes in requirements or priorities.

3.2.5 Sprint Retrospective

The overall goal of the sprint retrospective is to further increase the quality and the overall effectiveness of the upcoming sprints.

During the sprint retrospective, the scrum team reflects upon the last sprint and all the elements that led to the conclusion of that sprint. The scrum team then discusses all the different aspects of the sprint, what went well during the Sprint, what problems it encountered, and how the scrum team handled the issues.

The scrum team then concludes which of these changes has had the greatest increase in the effectiveness of the team and possibly adds it to the Sprint Backlog for the upcoming Sprint. The sprint retrospective is the last part of the sprint events and concludes the given sprint [21].

3.3 General Concepts

This section will focus on the artifacts present in Scrum, as well as their pertaining commitment, as detailed in the Scrum guide [21]. Within the work of Scrum, and the value that is being provided by Scrum, Scrum artifacts exist to maximize the transparency of key information. Each artifact contains a commitment. The commitments provide value to the Scrum team and stakeholders.

- Product Backlog
 - Commitment: Product Goal

- Sprint Backlog
 - Commitment: Sprint Goal
- Increment
 - Commitment: Definition of Done

3.3.1 Product Backlog

The product backlog is a list of items needed to further improve or develop a given product. These items often go through extensive refinement, which results in these product backlog items, PBIs, being broken down into smaller items, thereby making their implementation and purpose clearer to the developers, who are ultimately the ones who will be working on these items. The process of refining a PBI is not a one-time endeavor: the refinement is initially done mostly by the product owner, but they do not necessarily have any practical experience or knowledge when it comes to actually implementing these features. That is why it is perfectly acceptable and encouraged for the developers to come up with suggestions and refinements to improve the PBIs [21].

The commitment for the product backlog is the product goal, which serves as a target, or an end state for the product, that the entire Scrum team works towards at all times. It is the basis for every decision made when it comes to the creation and refinement of the product backlog [21].

3.3.2 Sprint Backlog

The sprint backlog is an assortment of concepts that collectively make up a plan for the developers, all to fulfill the sprint goal. This plan consists of three main concepts: the sprint goal, which is what needs to be accomplished during the sprint. The PBIs for the sprint, act as representations for the developers of the work needed to complete the sprint goal. And lastly, the increment, which is the actual work that has been completed during the sprint.

Upon starting a sprint, the plan may not necessarily be fully developed. As the sprint progresses, so does the Scrum teams' knowledge, allowing for potential improvements to the sprint backlog along the way [21].

The commitment for the sprint backlog is the sprint goal. As mentioned above, it is what needs to be done in the sprint, be it concise improvements to the product, or work towards other improvements to the team or product, like improving teamwork, or having the product in such a state that it allows for it to be usable in some way. The sprint goal should always be at the forefront of decision-making, as every decision should be made to progress towards the completion of the sprint goal [21].

3.3.3 Increment

An increment is in the context of agile, a term that refers to the implementation of new functions or improvements to the product. Every sprint has to result in a releasable increment. All prior and future increments will need to be thoroughly tested and verified to make sure that they work together. If the increments do not work together, any form of value they would otherwise possess ceases to exist. All increments serve to make progress towards the product goal. If work in an increment fails to meet the definition of done, it cannot fully be considered part of the increment [21]. This results in everyone being able to be held to a commonly agreed-upon standard across all groups [21].

4 Project Structure

4.1 Nexus Scrum Framework

The Nexus framework is a scaling framework used in more complex and large projects with more than one scrum team [25]. The Nexus framework makes it possible for multiple scrum teams to work together on the same project, helping with the difficulty of creating "done" code every increment due to the higher level of complexity of dependencies at scale. The Nexus framework helps with this by allowing organizations to plan, launch, scale, and manage larger product development initiatives using scrum [25]. Nexus allows multiple scrum teams to work on the same project to combine into a single larger unit, called a Nexus.

The framework can be thought of as an "exoskeleton" that helps scrum teams manage and simplify connections and dependencies among them and provides insight into how the different teams work together [26]. The foundation

of Nexus is communication and transparency to keep scaling as uniform as possible. As can be seen in Figure 4, the Nexus framework makes use of one product backlog used by all teams in the planning phase, with the sprint planning done in collaboration with all the teams [27].

In each sprint, the scrum teams within the project (referred to as the Nexus) conduct their work in their respective teams. However, the work done should be integrated with the other teams' work. This naturally raises a lot of issues, and it requires a lot of cross-team communication, coordination, and collaboration. To ensure that this goes as smoothly as possible, a Nexus Integration Team is established [27]. See figure 4, for a model illustrating the Nexus framework.

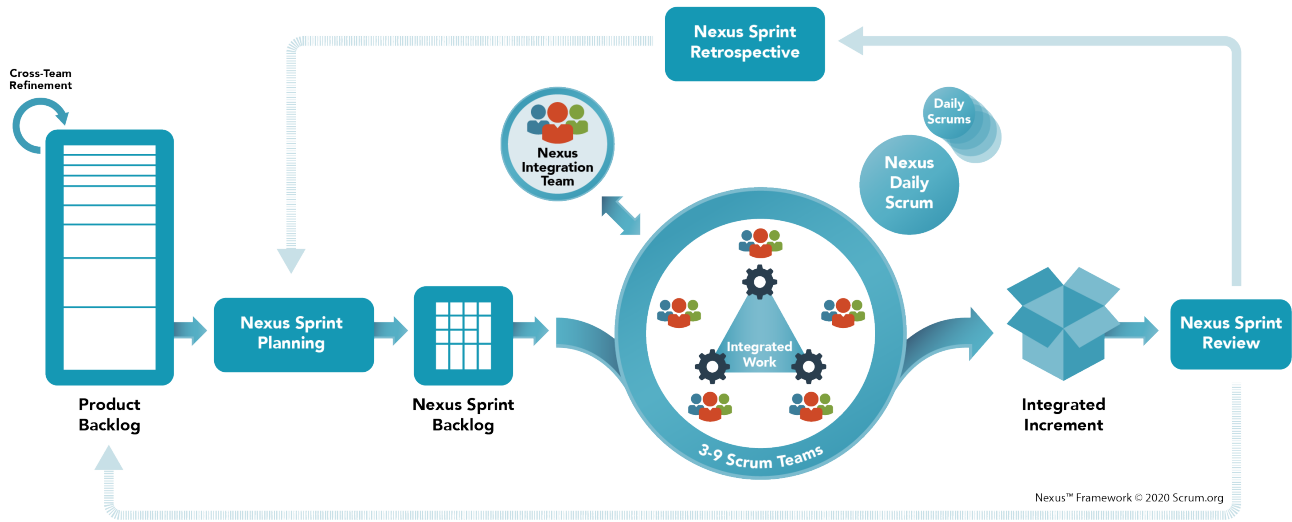


Figure 4: The Nexus framework [27].

4.1.1 Role and Impact of the integration team

The Integration Team should consist of the product owner, a scrum master, and team members. The Scrum master is responsible for ensuring that the Nexus framework is followed according to the guide, while the Product Owner is responsible for ensuring an effective management of the Product Backlog [27]. The team members usually ensure that tools and practices help the teams deliver a useful Integrated Increment [27].

In our project, the Integration Team was composed of a member from each scrum team, with one of these being chosen as the Nexus scrum master. The representing members could change, but not every team chose to rotate. As we had to be pragmatic with the limited time our PO had, we did not include them in all meetings, but rather when it was the most relevant.

4.2 Development process

This section will detail the development process and the different stages of the production pipeline.

Version control For the project, Git with GitHub was used to manage versioning. The entire project consisted of five primary repositories:

- Educado-frontend: The web application used by content creators for creating and uploading content to the Educado Mobile App.
- Educado-backend: The primary back end, is used for handling most of the traffic between the different parts of the system, like the mobile app, the web application, and microservices.
- Educado-mobile: The mobile app is responsible for serving the content to mobile users.

- Educado-transcoding-service: The microservice handles the transcoding of videos and all the traffic regarding GCP storage buckets.
- Educado-certificate-service: The microservice handles the creation, retrieval, download, and deletion of certificates.

Each repository consisted of three key branches;

- dev: The Development branch used for pushing new features that have been tested thoroughly and reviewed.
- staging: The Staging branch had changes pushed to it once or twice per sprint. All of these features should be stable and work as intended. This branch was used to present the product to the product owner in each sprint review.
- main: The Main branch, which acted like a production branch.

During the development and implementation, there were several branches for the different features. These branches would all have a prefix, depending on the group to which they belonged. For instance, "video-" would imply that the branch was used by the video streaming group. Then the name of the feature being worked on would be added. An example of a branch name could be "video-upload-to-bucket", which then further would imply that the feature implemented in the branch had something to do with uploading data to the storage bucket. This way, one could keep better track of the different branches.

When merging a branch into one of the collective branches (dev, staging, and main) we would create a new branch specifically for merging. This branch used the destination branch as base, and the merged feature branch as additions. This allowed us to more easily solve merge conflicts, as well as allow fine-grained control over which files you wanted to merge from the feature branch.

Code review When merging into any of the key branches, dev, staging, or main, the code had to go through code reviews on GitHub first. This was put in place to prevent faulty code from reaching important environments. Having multiple people review code helps in detecting any potential errors as well as ensuring that the quality of the code is sufficient. Therefore, it was agreed upon that at least one person, preferably two people, had to approve the changes for them to be merged into the desired branch. A simplified version of this structure is illustrated in the model below.



Figure 5: Pipeline.

This meant that only the dev branch could merge into staging, and only staging could merge into the main branch. As such, the only entry into this line was through the development branch. Since all these three branches demanded code review and testing, it provided a great number of opportunities to make sure any issues and bugs would be remedied upon traveling through the pipeline.

In practice, the branching structure is a little bit more complex. The model seen in Figure 6, illustrates how feature branches were used as well as merging branches.

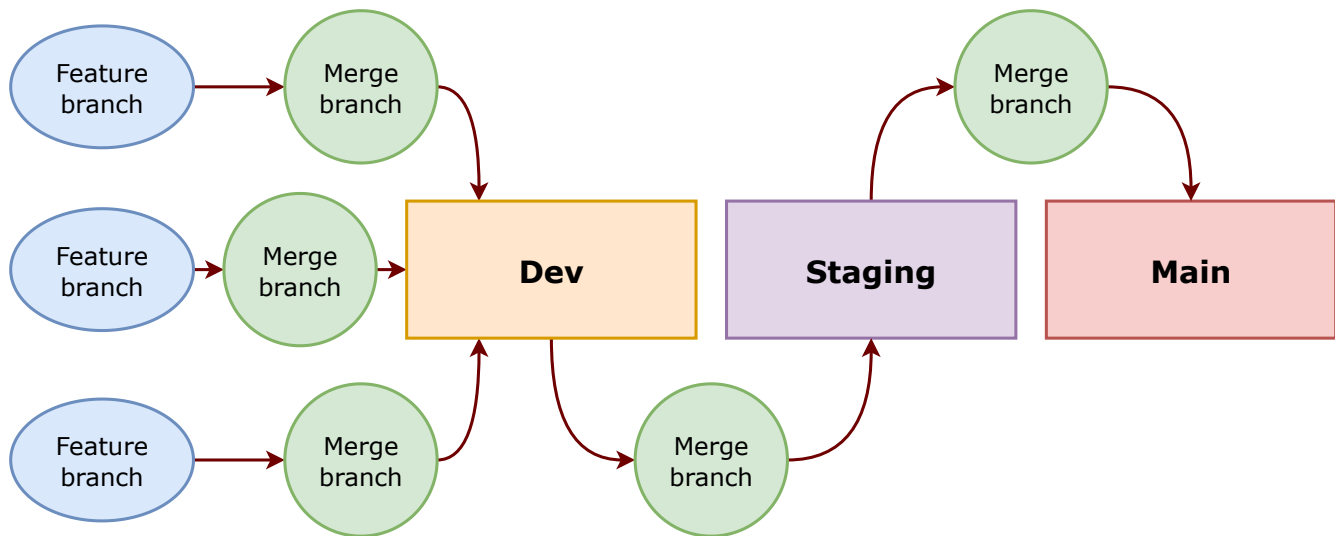


Figure 6: Model showing the branching structure of the project.

CI/CD When it comes to the continuous integration and the continuous deployment of the project, GitHub Actions were used for the 'Educado-mobile' and 'Educado-frontend' repositories, while the 'Educado-backend' is set up with Google Cloud. Different GitHub Actions workflows were used for the different repositories.

The 'Educado-mobile' repository has four different workflows set up.

- **Run Linting on Pull Requests and Branches:** Automatically performs linting checks on pull requests into dev, ensuring correct code style and props validation.
- **Run Tests on Pull Requests and Branches:** Runs the tests created on the branch related to the pull requests into dev.
- **CI:** Runs when a pull request is made to the staging branch. It runs both linting and automated tests. In addition, if all the tests pass, it builds an APK file. By building the app on all pull requests into staging, one can make sure that everything on staging is possible to build. In addition to identifying problems early, one can also test the APK files on Android devices before deployment.
- **CI/CD:** This pipeline is run on all pull requests into the main branch. It runs both the linting and the automated tests. Then it builds the app, but as an AAB file. When the build has succeeded, it automatically deploys it into production on the Google Play Store. This pipeline can be seen in Figure 7

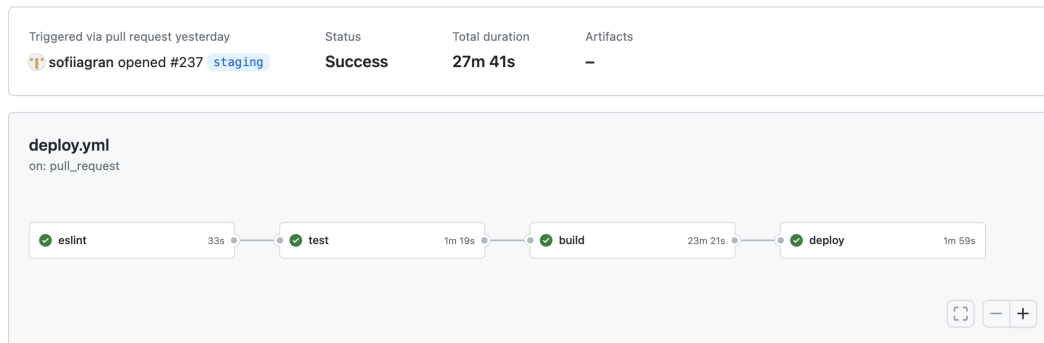


Figure 7: Model showing the CI/CD pipeline created for the mobile application on GitHub actions.

The 'Educado-backend' repository has two workflows: 'Run Linting on Pull Request and Branches' and 'Run Tests'. They are used for running linting and testing on all pull requests to dev, staging, and main. In addition, all pull requests that are merged into staging are automatically uploaded to the Google Cloud.

- **Run Linting on Pull Request:** Automatically performs linting on the Pull Request branch

- **Run tests:** Automatically runs the tests written in the branch related to the pull request on the development branch

Lastly, the 'Educado-frontend' repository had checks similar to the ones previously mentioned: type checking and testing. The front end, however, also has additional checks for building and deploying the web app using Netlify.

- **Type Checking:** Automatically performs type checking to verify and enforce any desired constraints on variables used in the code
- **Run Tests:** Automatically runs the tests written in the branch related to the pull request on the development branch
- **Deployment and Building:** Automatically uploads and builds the website using Netlify build.

Collaboration and communication For cross-group communication, as well as communicating with the product owner, four have mainly been used four tools, each for different, but important purposes.

Miro has been used extensively throughout the project, mainly as a hub for the sprints, PBIs, and their criteria. Furthermore, it included the entire product backlog and was used to organize the process of distributing PBIs to all the scrum teams.

Figma was used as a central hub for the designs of the web and the app. Collaborators from Brazil would design the UI for the web and the app for Educado and provide a clear design they wanted in Figma, as a guideline for the teams to follow.

Discord was utilized for general communication in and between the different scrum teams. It was the main platform used for communication and planning. It was also used to share documents, files, and other forms of information.

GitHub was the platform used to host the code base, as well as hosting the different features each group was working on in a given time frame. GitHub also allows for easy communication with pull requests and offers users a section to report issues with any given code.

5 Clean Architecture

Clean architecture is a software design philosophy that separates aspects of software development [28]. It prioritizes maintainability, testability, and scalability. Clean architecture follows object-oriented design principles and is often used in conjunction with agile development methodologies [28].

One of the benefits of clean architecture is its ability to make software maintenance easier as it grows and evolves [29]. By separating business logic from infrastructure, individual components can be upgraded without impacting the code-base. This allows for testing components in isolation, which is crucial for ensuring high-quality software [28]. Clean architecture also promotes modularity and code reuse, enabling developers to create libraries and modules by breaking down software into independent components that are easily integrated into other projects. However, like any software development philosophy, clean architecture has its limitations - implementing it can be time-consuming in large-scale projects. Finding the balance between modularity and complexity can also be challenging, leading to an eventual difficulty in maintaining designs [29].

In the Educado project, it was attempted to apply the principles of clean architecture in the repositories that contribute to the building of the platform and develop additional microservices, which support the clean architecture model to varying degrees. As an example, every time a request was made to the back end, it was sent to the back end route index, which determines what document the correct route is located in. These routes further need to access the different database models, which are sectioned accordingly and accessible where needed. However, there are also examples of code repetition and some components that are not dynamic nor scalable, to the extent that they are living up to clean architecture principles. There are also examples of functionalities that were not as modular and layered as described by the principles of clean architecture. While one would always prefer to utilize clean architecture, the main purpose was to deliver valuable increments for each sprint, and therefore, clean architecture was not the main priority at all times.

6 Microservice

In modern software development, using microservices is becoming increasingly popular among developers. A microservice is a small independent piece of software that has a specific functionality within a larger system. Microservices are quite different from the monolithic architecture, where all components in the application are written within the same unit or repository [30]. Microservices operate by themselves and interact with each other across APIs. Using microservices allows each service to be developed and launched independently, which allows for better continuous development [30]. This section of the report will dive deeper into the concept of microservices, and discuss its benefits as well as its general impact on the software development process, along with a discussion of our use of microservices in this project in Section 7.1.1

The adoption of microservices in software development is driven by a need for more scalable, flexible, and maintainable systems. Microservices offer numerous benefits, including improved scalability, where each service can be scaled independently based on demand; enhanced flexibility in using different technologies and frameworks for each service; and easier maintenance due to the smaller, more manageable codebases [31]. However, implementing a microservice architecture is not without its challenges. It can introduce complexity in system coordination and communication, the potential for increased resource consumption, and the need for comprehensive strategies in continuous integration, deployment, and monitoring [31].

6.1 Technology Stack and Tools

In the technology stack for microservices, Docker emerges as a central tool, widely recognized for its effectiveness in creating, deploying, and running microservices [31]. Docker provides lightweight containerization technology, allowing each microservice to be packaged with its environment and dependencies, ensuring consistency across different development, testing, and production environments. This containerization is crucial for the independent deployment and scalability of microservices, as each service can be deployed, updated, or scaled without impacting others [31].

6.2 Communication standards

Synchronous communication is often implemented through HTTP/REST APIs. In this model, a client service sends a request to another service and waits for the response. This approach is straightforward and familiar to many developers, but it can create tight coupling and latency issues, as the client service is idle until it receives the response [31]. Our certificate service mostly uses this approach, where the microservice is accessed directly from the client.

Another important aspect of communication in microservices is the use of API gateways. An API gateway acts as a single entry point for all client requests, routing them to the appropriate microservices. It can handle cross-cutting concerns like authentication, SSL termination, and rate limiting, thereby simplifying the client implementation [31]. The back end acts as a sort of API gateway for the video streaming microservice - and could likely be used for more microservices in the future.

7 Sytem Architecture

7.1 The Educado platform architecture

The Educado project attempts to follow the microservice architecture. This means that the project consists of many small and independent services, which is an ideal way of developing complex projects.

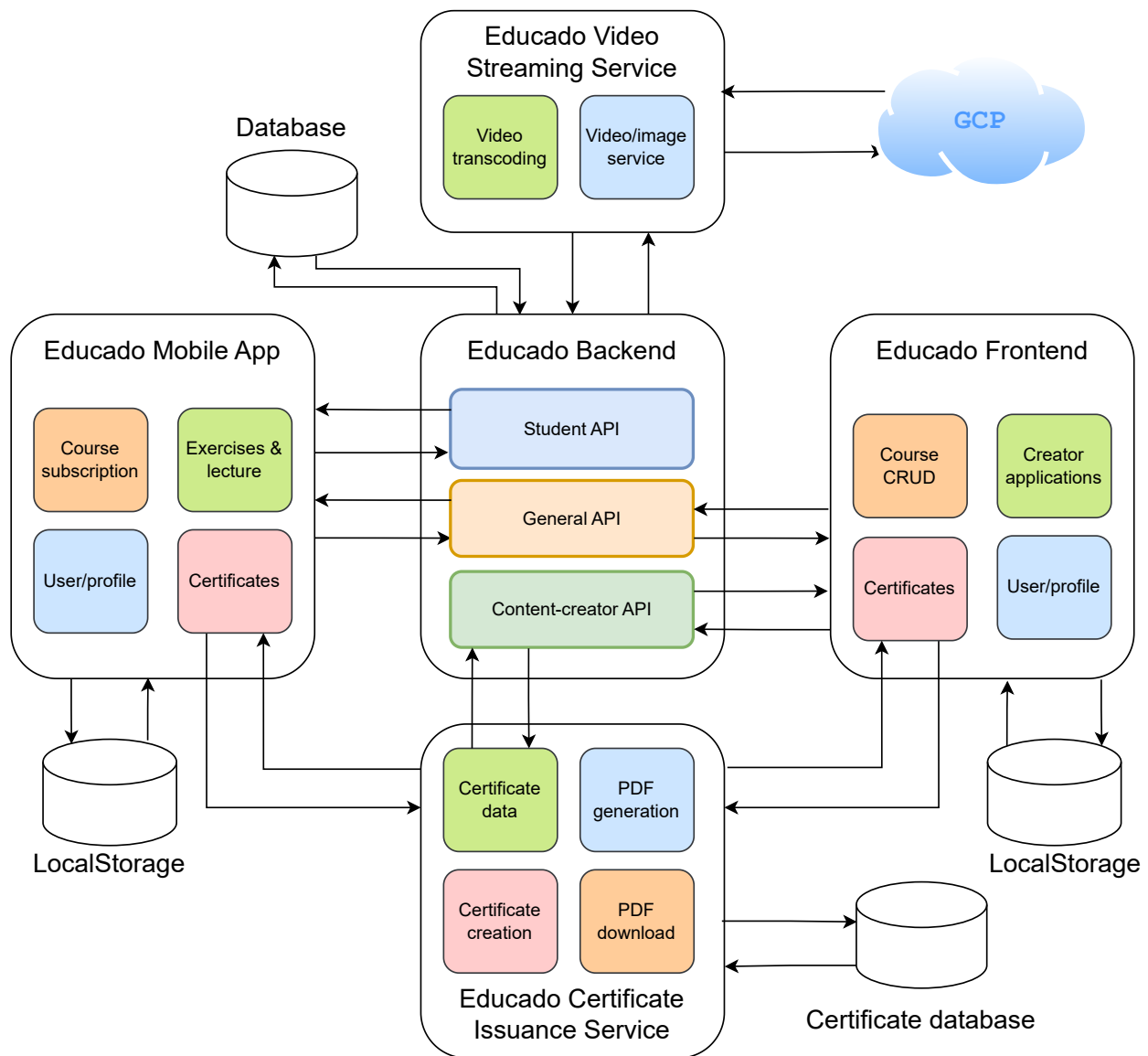


Figure 8: Architectural diagram of the Educado platform, consisting of three main repositories and two microservices.

Educado-mobile contains the front end for the mobile application, to be used by the waste pickers, while 'Educado-frontend' contains the front end for the web application to be used by content creators. These two microservices share a back end, which is the 'Educado-backend' microservice. This microservice contains the Student APIs, Content Creator APIs, and more general APIs. The Student and Content Creator APIs are specific to their platform (either mobile or web), while the general APIs are services that both the app and web use, such as login. Each platform has a way of storing information. The back end has access to the cloud database, while the web browser for the front end, and the smartphone for the mobile each have their local storage. This local storage allows the application to save data on the respective device for either authentication purposes, offline access, or fast data retrieval without communicating with the database. As seen in Figure 8, the back end communicates back and forth between the mobile and front end microservice, but the Educado project also has two other microservices, these being:

- Educado Video Streaming Service
- Educado Certificate Service

7.1.1 Educado Certificate Service

This microservice is in charge of transmitting, creating, deleting, generating PDFs, and providing the possibility of downloading certificates for both content creators and students. As seen in Figure 8, the certificate microservice is

both ways between the Content-creator API in the back end and the Certificate data in the certificate microservice. This is related to the content creator certificates, transmitted both ways between the Certificate microservice and the front end, as these contain dynamic information, and therefore depend on the back end as well. The reason for this is that content creators should be able to generate certificates at all times, after publishing a course, and therefore the data (such as how many completed the course) needs to be dynamically updated, with this information being transmitted from the back end.

As for the mobile app, students are only issued a certificate upon completing a course and therefore do not need dynamic data (at this stage of the project), hence no need for interacting with the back end for this matter. This means that actions related to the student certificates can be completed directly between the mobile app and the certificate microservice, without explicit need for the back end. The certificates are stored within their database (though in the same cluster as the general database), in case the certificate microservice encounters problems and shuts down, the rest of the Educado information is still accessible.

7.1.2 Educado Video Service

This service handles streaming videos, downloading media, and uploading images and videos in the GCP bucket. As shown in Figure 8, and more in-depth in Figure 9, the app interacts with the transcoding service through the back end, which acts as an intermediary. The back end essentially forwards requests and responses between the app and the transcoding service.

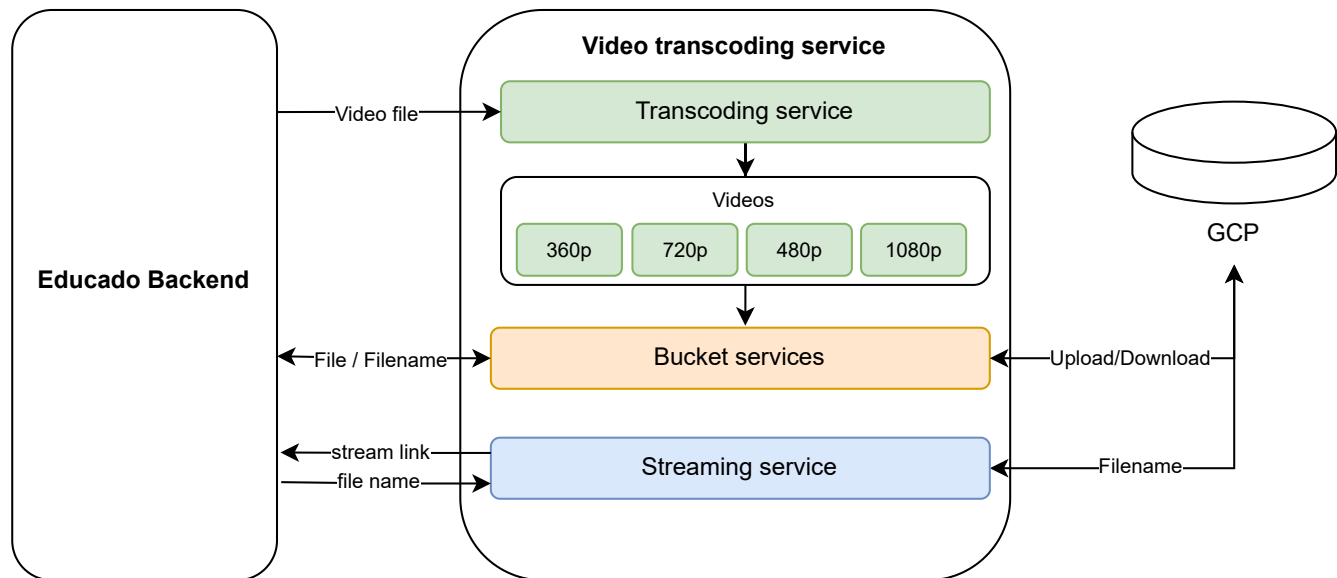


Figure 9: Architectural diagram of the video transcoding service.

The service is essentially split into three parts: bucket, stream, and transcoding. In an ideal world, one might make a fully functioning microservice for each part; however, in this case, they are under one hood. The bucket service handles uploading, downloading, deletion, and listing of all elements on the bucket. Images and videos are converted to base64 when fetching through the service, but apart from this, none of the data is touched. Since the mobile app needs to be served an actual video stream, and not just a base64 encoded video, the streaming service was created to be able to read a video file from the bucket and immediately stream it to the app, byte by byte. This makes for a smooth experience since the mobile app can play directly from the stream and does not need to wait for the entire video to be downloaded.

The transcoding service's purpose is to take a video as input, split it into 4 different resolutions, these being 360p, 480p, 720p, and 1080p, and then upload these videos to the bucket. This makes the end user able to choose a quality when streaming, thus making sure most people can stream, regardless of the quality of their internet connection. For our microservice, we opted to go with the programming language GoLang since it is especially good at handling concurrent tasks, which we found would be very useful concerning video transcoding. In our transcoding service, Go routines ended up having a large, positive impact on the overall system; the ability to handle multiple video

transcoding tasks concurrently was essential for making the service efficient and reliable.

7.2 Architecture reflections

In terms of the certificate issuance microservice, the sprints of this semester did not focus on the security aspects of the certificates. Therefore security has not been a priority for the certificate microservice. In the future, this could result in a dependency on the back end microservice, as certificate operations would likely require some form of authentication. This lead us to wonder if the architecture would benefit from authentication being a microservice on its own, rather than being a part of the general back end. This would be a step to further embrace the microservice architecture, and allow dependencies on only the authentication, rather than the back end, which has several functionalities other than security.

Overall, the use of Go and its Go routines was an important factor in the success of our transcoding service. The ability to handle multiple tasks concurrently, combined with easy-to-read code, made Go an ideal choice for our project. However, it would have been even better, if we had been able to split the entire service into its three subparts. That way, we would have followed the microservice architecture even better, by distributing everything out.

8 Data Models

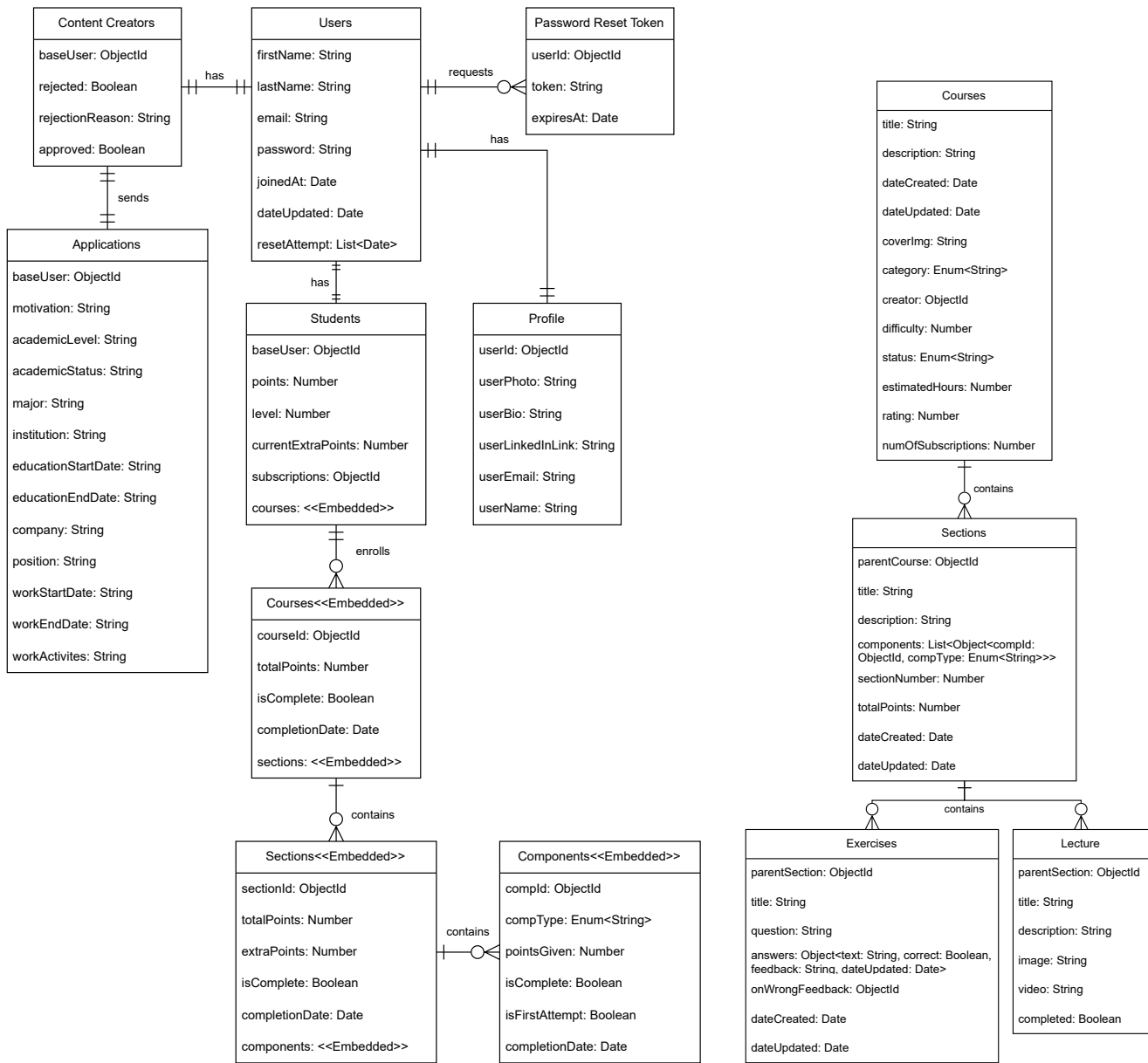
The finalized data model for the project will be discussed in this section. The visualization of the data model is made as an ER diagram, although MongoDB is a NoSQL document database and not a relational database. The database diagram has been split into two clusters for simplicity. The first cluster contains the Users, and the second contains Courses. The students connect the clusters using the Courses and the Content Creators create the Courses.

The first cluster shown in figure 10a is the user cluster showing the relations between the user model and its children. The first relation to the users is content creators, which must be approved by sending an application. A model for institutions of the content creators was also created, however, this is omitted in the diagram, as it does not have any references to other models at this stage.

The second relation of the users model is students, which is used to represent mobile users. The student model has a relation to courses, which is all the courses in which the student has either attended a lecture or answered an exercise. The course field within the Student model is an array that holds ObjectIds for each course that the student has taken. The management of course enrollments and student progress is handled within each course. Each index in the courses array has a courses object that also contains an array called sections. The indices of the sections array contain each section of the course. The sections contain a components field, which is an array of components for the specific section. This can be either exercises or lectures. When the student subscribes to a course, the course is added to the courses field in the student model. A component can either be an exercise or a lecture. Lastly, the third child of the user model is the profile which is used for web users. Another collection connected to the users is the password reset token, which is the token used to reset the user's password.

The second cluster in figure 10b is an overview of the relations between the Courses and their children. The course model has a list of ObjectIds which is a reference to all the sections in the course. The same principle is applied in the section model, where the only difference is that components are a list of objects with the ObjectIds stored in compId, and its type is also stored in compType. The compType property is used to determine if a component of the section is an exercise or lecture.

Furthermore, we use a database for the certificate service, although this is very simple and does not contain any direct relations (although it does have the IDs for the user and course associated with the certificate). As such, a diagram for this database has not been included.



(a) The user cluster of the database diagram

(b) The course cluster of the database diagram

Figure 10: Two ER diagrams for users and courses respectively in the database. Note that the implementation of relations may not be exactly as shown in the diagram since we are using a NoSQL database.

9 Sprints

This section will run through all the sprints that the group has worked on during this project. During the project, there were a total of 6 sprints, each lasting an estimation of 2 weeks. For this type of project there are certain terminology that has been used to describe specific elements of the project. These terms are important as they have been a guide throughout the project, and the groups' understanding and use of them in their structure will shortly be described here. The sprints will have the following structure:

- Introduction: Overview of the sprint, general information about the sprint, and shared sprint goal.
- Planning: The planning of the sprint, and a list of PBIs to be completed in the sprints' time frame.
- Daily scrum meetings: A Meeting the group has had each day of the sprint, discussing what each pair has been achieving and what they plan on doing next.
- Integration team: The discussions and findings of the integration meetings which is held by the integration team throughout the sprint.
- Increment: What the group has implemented and achieved throughout the project.
- Review: Presentation for the PO of the group's progress, then they receive Review feedback of their work, and then acceptance or rejection of the PBIs the group has presented.
- Retrospective: Reflection of the group's work and work process, reflection of the teamwork between the groups and overall look of the outcome of the sprints process, and further improvement.

9.1 Preparation prior to the sprints

Before the teams started the first sprint they all met and got an introduction to the product owner (PO). The product owner represents the target group which in this project is the waste pickers in Brasilia. The product owner gave all the teams an introduction to the Educado project so all the teams had the same background knowledge about the importance of the project.

The teams were also given a presentation, from two previous developers to the already existing code which was made by the prior semester teams. With an introduction and presentation into the project the teams were ready to begin their first sprint in the Educado project.

9.2 Sprint 1

Duration of sprint: 11/09/2023 - 25/09/2023

The first sprint started on the 11th of September 2023, this sprint focused on getting the project started. This requires research of the code, which enables the group to create an understanding of the project, project scope and the functionality of the current iteration. The groups also have to focus on the given task and find a constructive way, to solve the issues in this fairly new application.

In the beginning of the sprint, the PO described the spring goal to be: *"Stabilize and enhance existing features of the Educado platform, focusing on design improvements, usability fixes, and performance optimization. Prepare the foundation for future feature integrations while ensuring a robust and user-friendly experience for both content creators and learners [32]."*

9.2.1 Sprint 1 planning

At the start of the first Sprint of this project, the groups were presented with a product backlog containing items (PBI's) in the form of user stories, furthermore an indicator color "pink" if a PBI has been started by previous developers, or "blue" if not. The entirety of the groups had the responsibility to assign a complexity level of development to the PBI's, the complexity scale being, from "S" (lowest) to "XL" (highest), or "???" for unknown. The scale is also sometimes represented in a numerical manner "1" (lowest) to "5" (highest) or "0" for unknown. After the complexity had been assigned by the class, the product owner assessed and assigned values to each PBI ranging from "100" to "10,000", where a higher value reflects more importance. All the groups had to choose between the PBI's, and in the end of the choosing process, this group ended up with a handful of PBI's that relate to one another.

PBI 1:

[Web Create] As a content creator, I want to be able to put information on the new section of the new course I'm creating - difficulty 2, value 5,000

PBI 2:

[Web Create] As a content creator, I want to be able to add a section on the new course I'm creating - difficulty 3, value 500

PBI 3:

[Web Create] As a content creator, I want to be able to put a lecture on the section I'm creating - difficulty 3, value 500

PBI 4:

[Web Create] As a content creator, I want to be able to put general information on the new courses I'm creating - difficulty 3, value 500

PBI 5:

[Web Create] As a content creator, I want to be able to put an exercise on the section I'm creating - difficulty 4, value 1,000

9.2.2 Daily scrum meetings

In prior projects, the group has had morning meetings every day, where each pair within the group presented what they achieved the previous day, and what they plan on doing throughout the day. This way of meetings fits perfectly with the daily scrum model. Therefore the group just continued doing these daily scrum meetings in the morning and keeping track of their progress in their logbook. Keeping up with the daily scrum meetings was not an issue for the group. However, the time that the meeting was conducted was not systematically the same each day which could be improved in future sprints.

9.2.3 Integration team

As a part of the Nexus framework an integration team had been created, this integration team consists of a group member from each group across all the groups working on this project.

The integration team had two meetings throughout the first sprint, the first one determining the standard formalities that all the groups had to follow. Most standards followed the same as last year, such as using React, Typescript, and Tailwind in the frontend, and using JavaScript, NodeJS, and Express in the backend. In the second meeting, tasks were delegated to different groups. E.g. one group had to change the storage from using Amazon Web Services (AWS) to using Google Cloud, while this group started to look into GitHub Actions.

The integration team formulated these rules for the first draft of the definition of done:

- All code must adhere to the coding standards agreed upon.
- Code must not contain any dead code or unused imports.
- All designs must follow the design guide on Miro.
- Acceptance criteria must be fulfilled for a PBI to be done.
- Use the extension to Visual Studio Code called "Doxygen" to write comments.

9.2.4 Increments

Throughout the first sprint of the project, none of the groups had the keys required for Mongoose, necessary for running the code. Without the keys, it was only possible to see a loading screen because the code was reliant on data, and the groups could not log in. Therefore, during the majority of this sprint, the group could not make any progress regarding the code. However, the group used this time to gain insight into the existing code and thereby prepare for their own PBIs when they gained access.

At that time, one thing worth considering was some design choices. The group was responsible for creating and modifying the design for the website until the group received a design guide from the designers in Brasilia. Before

coding, the group communicated with the product owner to establish a consensus on the design for the "create course" pop-up. The group made the following paper prototype for easy modifications:

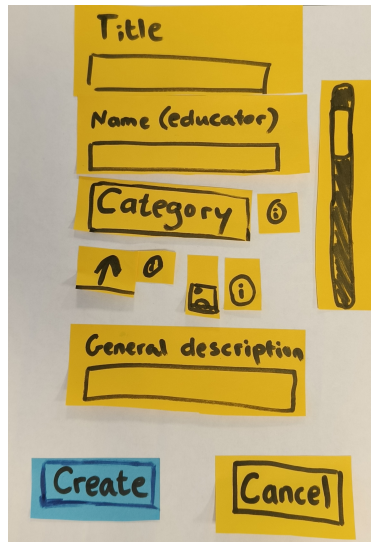


Figure 11: The first draft of a design for create course [22].

The product owner was overall happy with the design but had a few suggestions for changes, such as the arrow shown in the picture should be replaced with the Portuguese word for level and the name of the educator was determined to be unnecessary. The group added a suggestion that was not originally from the PBI, which was information icons beside different buttons that, when clicked, should show a description. The product owner thought that it was a good idea, as it would add to the understanding of the fields in the pop-up.

9.2.5 Sprint 1 Review

When looking at the group's project from the perspective of the given PBIs in this sprint, it can be concluded the group has not been able to make much progress when looking on the given tasks. This has been due to the workload of researching the current code functionality, packages, and front-to-backend capabilities. Furthermore, during the process, the group was working on making a functional version of the front and backend, which would give them insight into how the code corresponds to the visuals, however, this was not straight off the bat. This was largely due to the currently given keys to the backend being incapable of allowing the groups access to the backend database.

The group chose a different person to be the Scrum Master for each sprint, however, the role was not that crucial because each group member was very independent throughout the whole process. At the beginning of each sprint, the PBIs were distributed among the team members, making the responsibilities of each group member clear.

9.2.6 Sprint 1 Retrospective

After receiving feedback from the product owner the group had to reflect on how this sprint went and what has to be changed for the next sprint.

What went well In this sprint, the group has used a lot of time getting familiarized with the code and, therefore, has gained a deeper understanding of the project they have been given.

What did not work Throughout this first Sprint, all of the teams experienced a lack of cross-team communication and, therefore, an unnecessary amount of confusion both internally within the group and externally between the other groups occurred.

What has to be changed Into the next Sprint, cross-team communication has to be heavily improved so the development can run more effectively and smoothly. In correlation to that, the group should improve their time management of their PBIs, if dependencies occur the team should be better at improvising and making what is

possible in the given setting. Another thing that was pointed out at the sprint review was that everything on the website and in the app must be written in Portuguese, and not English, as it is important that it is easily understood by the users who primarily speak Portuguese.

9.3 Sprint 2

Duration of sprint: 25/09/2023 - 09/10/2023

After the first sprint, it is time to start the second sprint in the Educado project, this was started with the planning of Sprint 2 for all the teams and the product owner. The goal of Sprint 2 was *to focus on enabling the web course creation and mobile app access in the Educado platform. This has to be done while focusing on design, usability, performance improvements and user-friendliness [32]*.

9.3.1 Sprint 2 planning

The group got the chance to continue working on the same PBIs as they did in the previous sprint, this was decided in cooperation with the product owner. However, even though the PBIs were the same, some changes in their acceptance criteria were made. The group's PBIs for this sprint are:

PBI 1:

[Web Create] As a content creator, I want to be able to put information on the new section of the new course I'm creating - difficulty 2, value 5,000

PBI 2:

[Web Create] As a content creator, I want to be able to put an exercise on the section I'm creating - difficulty 4, value 1,000

PBI 3:

[Web Create] As a content creator, I want to be able to add a section on the new course I'm creating - difficulty 3, value 500

PBI 4:

[Web Create] As a content creator, I want to be able to put a lecture on the section I'm creating - difficulty 3, value 500

PBI 5:

[Web Create] As a content creator, I want to be able to put general information on the new courses I'm creating - difficulty 3, value 500

9.3.2 Daily scrum meetings

Throughout the sprint, the group held their daily scrum meeting. In this sprint, they discussed how to plan their work on the project between the lectures of the courses in the semester. The group members were a bit concerned about how to place their time between working on the project to be sure to complete their PBIs and working on exercises for their courses. On the days that the group had a lecture in the morning, the group would focus on their lecture from 8 o'clock to 12 o'clock, and then they would work on the project the rest of the day.

9.3.3 Integration team

In this sprint, the integration team has had a lot more meetings than in the first sprint. This resulted in better communication between all the groups, and common issues could be looked at and discussed by multiple groups. The group experienced a lot of issues with some 'buckets' in the code regarding the possibility of uploading videos or images. This was discussed at the integration meeting, and it was discovered that one of the other groups, the Video Streaming Group, had a similar issue with buckets on another PBI. The group had to make buckets communicate correctly with the frontend, while the Video group had to make buckets communicate with the backend. The two groups ended up collaborating and sharing knowledge on the issue, thanks to the more frequent integration meetings.

During the first integration meeting of this sprint, it was discussed that someone should be in charge of setting up

some GitHub Actions, which all the teams could use to report issues to each other if needed and make the process more automated. It was decided that members of this group were placed as responsible for setting them up. The integration team also discussed the assistant teacher/ product manager's time limitations since he doesn't have the opportunity to be available at all times. All questions should go through the integration team in a designated time frame.

Based on this interaction, the group would deem that the use of integration meetings has been far more beneficial and used more properly in this sprint than in the previous one.

9.3.4 Jest testing

Jest was introduced early on for both the frontend and backend repository testing. Jest was chosen as it works very well with Javascript and Typescript [33], which the backend and frontend repositories use, respectively. Some Jest testing for the backend was already implemented by the students working on the project last year, which made it easier to introduce to the groups this semester, as well as Jest specifically being made for testing with the react library [34].

Snapshot testing [35] was used for the frontend testing, while asynchronous testing [36] was used for the backend testing. The frontend repositories didn't have any testing when the groups started working on the project. This resulted in the groups having to learn how to test using the jest framework for design and end-to-end testing on the frontend. This caused quite a bit of trouble, as testing frontend with jest was not as simple as testing the backend, as the references for frontend used javascript instead of typescript, which frontend was built with. Quite a few groups had also trouble with snapshot testing, as any change in the design would result in errors in testing if the snapshots were not updated.

The snapshot testing through jest was later replaced with Cypress, which will be discussed further in sprint five section 9.6.2.

9.3.5 Increments

In the second sprint, the group members continued with the PBIs they previously worked on during the first sprint. Clear goals were formed as the group shifted to a pair-programming approach of smaller groups of two focusing specifically on their PBI. Since the group members already had a comprehension of that part of the code.

One pair in the group focused on the PBI:

[Web Create] As a content creator, I want to be able to put general information on the new courses I'm creating.

However, the group encountered many issues with the buckets in this PBI. The buckets handle the upload, saving, and loading of images, such as the cover image of the course - which is a part of the acceptance criteria for that PBI. The pair used a great amount of time and effort to fix this issue and also collaborated with the Video Streaming group, as mentioned in the previous section *Integration team 9.3.3*. The group, however, encountered issue upon issue with the buckets, and therefore, the PBI was not completed in this sprint. However, the bucket issue was the only part of this PBI that kept it from being completed, as the pair managed to fulfill the rest of the acceptance criteria for this PBI. The pair also found an issue with the course page not showing on the website. With further inspection, it was discovered that there was an issue with the authentication token, which was needed for the page to work correctly. The authentication token was not created by the group, and therefore, they communicated the issue to the group responsible for that token. In the meantime, the pair found a workaround solution so they did not become dependent on waiting for the responsible group to solve that issue.

Another pair within the group focused on the following PBI:

[Web Create] As a content creator, I want to be able to add a section on the new course I'm creating.

This PBI took a great deal of time to complete as well even though it only had two acceptance criteria: *"Sections page with a button 'add new section' and 'Translate the page to Portuguese'".* The group encountered many issues with the question of what the best way was to reference the section, should it be through the course and its course id which the section was created under or through an id connected to the section. The group decided to give the section its own ID to make their future PBIs regarding adding lectures and exercises to the section easier however, the page should get the section detail through the course it was created under. While working on this PBI, the group also made it possible to give the section general information such as title and description for future use, therefore also solving the PBI:

[Web Create] As a content creator, I want to be able to put information on the new section of the new course I'm creating which had the acceptance criteria: Input text: section name (Mandatory, if empty display error) and Input text: description (NOT Mandatory).

9.3.6 Sprint 2 Review

Throughout this sprint, the group was able to work on the following PBIs:

[Web Create] As a content creator, I want to be able to put information on the new section of the new course I'm creating

[Web Create] As a content creator, I want to be able to add a section on the new course I'm creating.

[Web Create] As a content creator, I want to be able to put general information on the new courses I'm creating.

Until now, including this review, the groups made their presentations for the sprint review individually without much coordination. One group presented their increments in a PowerPoint, which all groups agreed should be the new standard going forward. It was pointed out that the presentations should be smooth and unified, and only things relevant to the stakeholders should be shown. The work done on these PBIs was presented to the product owner at the sprint review and gave feedback. The product owner liked the work done and agreed that these PBIs were completed but still were missing the buckets/ upload cover image function. The PBIs were also not up to date on a style guide, but that was because a style guide was not created yet for these pages, it was therefore decided that new PBIs would include updating to a style guide when it was created. The product owner also, to the group's surprise, revealed that the group had by accident, solved PBIs that had not yet been given to any group:

[Web Edit] As a content creator, I want to be able to edit the sections of the courses I haven't published yet

[Web Home] As a content creator, I want to be able to edit the courses I haven't published yet

However, the edit course PBI also required that it should be possible to change the cover image, which is affected by the reoccurring bucket issue the group has experienced. Therefore the Create course and Edit course PBI were not determined to be complete, and the group would continue working on those in the next sprint.

The three other PBIs the group had managed to complete fulfilled the definition of done and were determined to be complete. They were, therefore, ready to be merged into staging.

9.3.7 Sprint 2 Retrospective

After a relatively rough first sprint, the group focused on improving communication and task allocation during this sprint. This happened through the formation of smaller groups and the prevention of overlaps. The group used inter-group cooperation, helped enhance automation through GitHub Actions, and found a better balance of responsibility.

What went well Having a more transparent workflow, and clear focus from the time the group gathers for work. Inter-group cooperation also greatly increased the effectiveness. But also, the approach from the last sprint, which was to just understand the code, greatly increased efficiency.

What did not work Throughout the second Sprint, the group found that there was a lack of structure or schedule, which led to some workload variation making it difficult to form a structure from the beginning of the day.

What has to be changed In the next Sprint, scheduling has to be improved so that the collective impact and focus are in sync. Resulting in an overall more effective and smooth workflow. Furthermore, the group wanted to try and incorporate more feedback with the Project Manager as making a continuous running conversation will establish an alignment of expectations. With the introduction of GitHub Actions, the group also plans to increase quality assurance through prioritization and testing. Also, the group will try to sustain the current inter-group communication as that was a great success. The group also has to communicate with the Video-Streaming group, who are responsible for creating the code for the backend buckets, since a big amount of time has been spent trying to make the buckets work in the front end. Perhaps the problem can be fixed more efficiently if the group looked into how they solved it.

9.4 Sprint 3

Duration of sprint: 09/10/2023 - 22/10/2023

The product owner described and stated that the sprint goal for this sprint is: *"To focus on enhancing the user experience and design on the web and app, this included optimizing and enhancing the course edit option on the web and improving the social-gamified experience on the app. It should also be a priority to prepare for the stakeholder validation and testing and ensure deployment readiness [32]."*

9.4.1 Sprint 3 planning

PBI 1:

[Web Create] As a content creator, I want to be able to put information on the new section of the new course I'm creating - difficulty 1, value 10,000

PBI 2:

[Web Create] As a content creator, I want to be able to put general information on the new courses I'm creating - difficulty 2, value 10,000

PBI 3:

[Web Create] As a content creator, I want to be able to add a section on the new course I'm creating - difficulty 1, value 10,000

PBI 4:

[Web Create] As a content creator, I want to be able to put a lecture on the section I'm creating - difficulty 4, value 10,000

PBI 5:

[Web Create] As a content creator, I want to be able to put an exercise on the section I'm creating - difficulty 3, value 10,000

PBI 6:

[Web Edit] As a content creator, I want to be able to edit the sections of the courses I haven't published yet - difficulty 1, value 10,000

PBI 7:

[Web Edit] As a content creator, I want to be able to edit or delete sections, lectures or exercises of the courses I haven't published yet - difficulty 3, value 5,000

PBI 8:

[Web Home] As a content creator, I want to be able to edit the courses I haven't published yet - difficulty 2, value 1,000

PBI 9:

[Web Create] As a content creator, I want to be able to save the course I'm creating as a draft, so I can finish it later - difficulty 2, value 200

PBI 10:

[Web Edit] As a content creator, I want to be able to edit the general info of the courses I haven't published yet - difficulty 2, value 10,000

9.4.2 Daily scrum meetings

For this sprint, the group chose a new Scrum Master who, as usual, had the responsibility to make sure that the group used Scrum. Every morning the work assignments were distributed among the team members.

9.4.3 Integration team

Near the end of the sprint, the integration team agreed that there was no need for unit tests in frontend, instead cypress and end-to-end testing should be used. This decision should help the efficiency to quicker getting code into

staging. Furthermore, the integration team discussed the importance of all groups contributing to reviewing code.

9.4.4 Increments

At the beginning of this sprint, some group members were working on new features, such as lectures and exercises, while others were working on merging the code made by the group in the last sprint into the development branch. Being able to put lectures and exercises in a section was given a very high value by the product owner, and therefore the group prioritized to work on those PBI's. However, in the last week of the sprint, all team members spent all the time only on getting code into the dev branch, as it turned out to be a quite complex task.

Before the code for creating an exercise was made, the group made a paper prototype, making it possible to discuss the design with the product owner before implementing it. In collaboration with the product owner, the group made the decision that there should be a maximum of four answers on an exercise and only one correct answer. The users should get feedback based on which answer they choose, so in the case of choosing an incorrect answer they have an opportunity to learn.

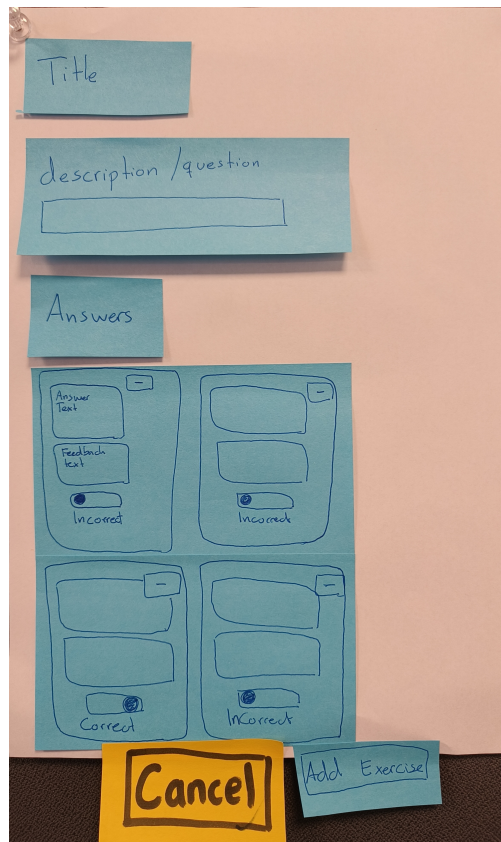


Figure 12: The first draft of a design for create exercise [22].

In addition to exercises, the group also worked on implementing lectures. When making the design for the lectures, the group decided to follow the same structure as exercises however, lectures should contain a title, a description and the possibility to add an image or video, as seen in the figure 13. The product owner saw the prototype and accepted the design, and thereby the group could proceed to implement the exercises and lectures as user-friendly pop-ups.

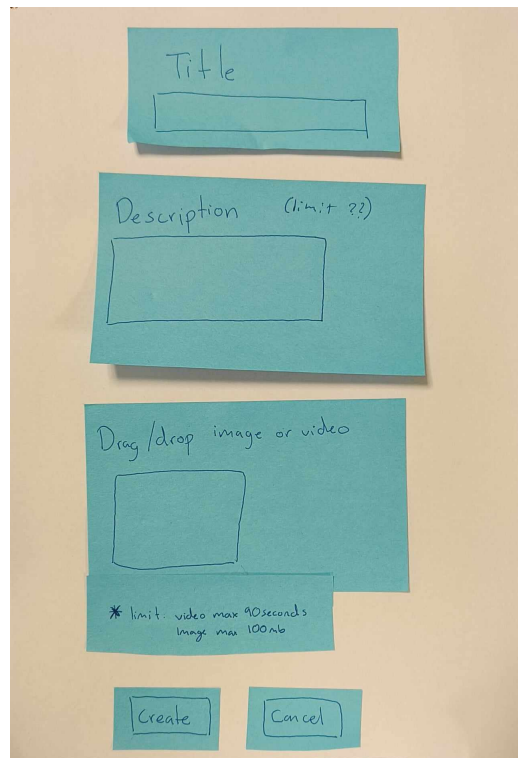


Figure 13: The first draft of a design for create lecture [22].

The group completed the coding for nearly all PBIs in this sprint, however didn't get it onto the dev branch. Nevertheless, the group did make a pull request however, there were too many merge conflicts, preventing it from being merged. Two PBIs were still missing the acceptance criteria of uploading a cover image, as the problem persisted from sprint 2, and therefore didn't live up to the definition of done. It concerns the following PBIs:

[Web Create] As a content creator, I want to be able to put general information on the new courses I'm creating.

[Web Home] As a content creator, I want to be able to edit the courses I haven't published yet.

Besides those two PBIs mentioned above, there were only three PBIs where the code wasn't yet completed. The group didn't finish the coding for these PBIs according to changes in the plan and prioritization as mentioned earlier. These PBIs are:

[Web Create] As a content creator, I want to be able to put a lecture on the section I'm creating.

[Web Create] As a content creator, I want to be able to put an exercise on the section I'm creating.

[Web Edit] As a content creator, I want to be able to edit or delete sections, lectures or exercises of the courses I haven't published yet.

The product owner suggested a meeting to show the content creators the progress of the product, but it was decided that it was necessary to postpone it since such a meeting would only be meaningful once the code had been successfully merged.

9.4.5 Sprint 3 Review

The groups had a more unified way of showing the increments in this sprint review, using the same PowerPoint template. The way the groups presented their PBIs had improved however, the sprint goal wasn't met. The assistant teacher and product technical manager suggested less strict requirements for frontend testing, as a way to ease the merging to staging.

Many groups experienced problems with getting code onto the dev branch of the project due to strict criteria from the definition of done, which required all acceptance criteria to be approved before being merged. This both delayed some PBIs from being merged but also created bigger pull requests which had to be reviewed by other groups and were a lot more likely to create merge conflicts. These merge conflicts further delayed these pull requests, as well

as most groups not delegating their time to review other groups' pull requests.

The product owner emphasized the importance of getting code into staging, to be ready for the showcase of the website in the coming sprint.

9.4.6 Sprint 3 Retrospective

In general terms, the group overestimated their own capabilities, the group communicated with the product manager in the sense that the group did not get the point across that they would not be able to manage the tasks they were given, the group also went in the wrong direction with pushing their functions onto the developer branch as the group delayed upload because there were other functions close to being finished. This resulted in the group's own merge to dev branch to outdate in more senses than one. Unfortunately, this meant that there were not any increments to show the users.

What went well The group managed to write code that would be beneficial for the coming sprint and familiarized themselves with the problems in getting the code into the dev branch which gave insight for smoother future implementations.

What did not work The pull request turned out to be too big, containing code from several PBIs, which made it very complex to get it into the dev branch, as there is a risk of more conflicting code. In addition, the size of the pull request made it more time-consuming to review by other groups, increasing the risk of delayed approval.

What has to be changed The group has to become better at openly communicating with the product owner throughout the sprint when realizing that the goal can not be met. This will make an opportunity to make changes that might help the process or, if necessary, change the sprint goal.

9.5 Sprint 4

Duration of sprint: 23/10/2023 - 5/11/2023

The product owner stated that the sprint goal for this sprint is a continuation of the last sprint's goal which was: *"To focus on enhancing the user experience and design on the web and app, this included optimizing and enhancing the course edit option on the web and improving the social-gamified experience on the app. It should also be a priority to prepare for the stakeholder validation and testing and ensure deployment readiness [32]"*.

9.5.1 Sprint 4 planning

Since the group realized that their initial estimations on how many PBIs they were able to complete were a bit too optimistic in the last sprint, the group agreed with the product owner not to receive any new PBIs for this sprint. Instead, the group agreed to refine and continue on their PBIs from the last sprint, with a focus on getting them onto staging and ready to showcase, since there will be a validation meeting on the website with the waste pickers in this sprint. In this sprint, the group will first and foremost complete the nearly finished PBIs, and when they are completely done, the group will work on the rest of the PBIs.

PBI 1:

[Web Create] As a content creator, I want to be able to put information on the new section of the new course I'm creating - difficulty 1, value 10,000

PBI 2:

[Web Create] As a content creator, I want to be able to put general information on the new courses I'm creating - difficulty 2, value 10,000

PBI 3:

[Web Create] As a content creator, I want to be able to add a section on the new course I'm creating - difficulty 1, value 10,000

PBI 4:

[Web Create] As a content creator, I want to be able to put a lecture on the section I'm creating - difficulty 4, value 10,000

PBI 5:

[Web Create] As a content creator, I want to be able to put an exercise on the section I'm creating - difficulty 3, value 10,000

PBI 6:

[Web Edit] As a content creator, I want to be able to edit the sections of the courses I haven't published yet - difficulty 1, value 10,000

PBI 7:

[Web Edit] As a content creator, I want to be able to edit or delete sections, lectures, or exercises of the courses I haven't published yet - difficulty 3, value 10,000

PBI 8:

[Web Home] As a content creator, I want to be able to edit the courses I haven't published yet - difficulty 2, value 10,000

PBI 9:

[Web Create] As a content creator, I want to be able to save the course I'm creating as a draft, so I can finish it later - difficulty 2, value 10,000

PBI 10:

[Web Edit] As a content creator, I want to be able to edit the general info of the courses I haven't published yet - difficulty 2, value 10,000

9.5.2 Daily scrum meetings

The group once again chose a new Scrum Master for the sprint. The group struggled a bit with keeping the logbook updated throughout the days, but this was fixed and a log for each day was created. Even though the logbook was not properly updated in the process, the group still succeeded in having daily scrum meetings.

9.5.3 Integration team

The group experienced some difficulties with a lack of communication between the other groups, this was taken up in the integration meeting and has since been improved further. The integration team also discussed which features should be prioritized to be tested during the online user test, since the focus for this test was on the web application and course creation, it was decided that creating a section, lecture, and exercise should be prepared to be tested. The login and create profile features should also be tested. However, the group was not responsible for these features, and therefore, they will not be described further.

9.5.4 Increments

In the first part of this sprint, the group focused on testing the front- and backend related to the PBIs. This took a couple of days, but all the tests came back successfully. Now the group had to create a pull request for their progress and get their PBIs into the dev branch and then staging for the showcase, this also took multiple days but lastly, the group got the PBIs into the staging branch. However, the PBIs created some issues on the server due to higher restrictions, these issues are looked at and will be looked at in future sprints as well.

In this sprint, there was a showcase for some stakeholders from UNB (University of Brasilia) who also are going to be the primary users of the website. The user test feedback, results, and outcome will be discussed in section 9.5.5.

The test users encountered a pop-up when creating a course, requiring them to input a title, category, image, level, estimated time, and description as seen in figure 14.

Figure 14: The pop-up for the creation of a new course.

9.5.5 User test

The group gained a lot of feedback from this user test. The feedback was discussed in a meeting with the product owner. A stakeholder from Brasilia tested the product through a video call. The group took notes which were relevant to them. The following notes to changes were these.

Course home page

- The description should be shown on the course navigation page.
- Option to see other content creators' courses or just their own created courses.

Course

- The upload file should be clearer on how and what to do - both in the update course and create a course.
- The button back to the homepage is not clear enough.

Section

- When creating a section, the create button should be more intuitive and changed.
- The section page needs more information on it.
- Needs more clear navigation.
- Create Lecture and Exercise button should be bigger and the colours more pronounced and a stronger background color.
- The button back to the course is not clear enough.

Lecture

- More options for creating lectures: Video, Image, Link.
- The Text should be bigger and stronger blue.

Create Exercise

- It is not very clear how it is set up
- More explaining text - instructions on the page on how to create it
- Needs a direct option to pick the number of answers to a question
- Written how many questions you can have

- The questions should appear on the right side so they can be seen at all times on the Create Exercise page
- Text should be bigger and have a stronger background color

The feedback was discussed with the product owner, and it was decided that the feedback should be reflected on new PBI's in the next sprint.

9.5.6 Sprint 4 Review

In the sprint review, the group presented their progress in this sprint. In this sprint, the group finally got their completed PBIs into the dev and staging branches. The process of getting the PBIs into the branches was longer and more difficult than expected and took most of this sprint. However, it was done, and with no feedback or changes from the product owner, the PBIs were marked as Done and are, therefore, completed. The completed PBIs are the following:

PBI 1:

[Web Create] As a content creator, I want to be able to put information on the new section of the new course I'm creating - difficulty 1, value 10,000

PBI 3:

[Web Create] As a content creator, I want to be able to add a section on the new course I'm creating - difficulty 1, value 10,000

PBI 5:

[Web Create] As a content creator, I want to be able to put an exercise on the section I'm creating - difficulty 3, value 10,000

PBI 6:

[Web Edit] As a content creator, I want to be able to edit the sections of the courses I haven't published yet - difficulty 1, value 10,000

PBI 7:

[Web Edit] As a content creator, I want to be able to edit or delete sections, lectures or exercises of the courses I haven't published yet - difficulty 3, value 10,000

PBI 8:

[Web Home] As a content creator, I want to be able to edit the courses I haven't published yet - difficulty 2, value 10,000

PBI 9:

[Web Create] As a content creator, I want to be able to save the course I'm creating as a draft, so I can finish it later - difficulty 2, value 10,000

The following PBIs were also reviewed and discussed with the product owner, these however are not completed but are still in progress due to various reasons. The reasons are listed below the PBIs. The group will continue with these PBIs in the upcoming sprint.

PBI 10:

[Web Edit] As a content creator, I want to be able to edit the general info of the courses I haven't published yet - difficulty 2, value 10,000

- This PBI is near completion, but the upload file acceptance criteria have not been completed due to difficulties and complications with the buckets required in the database code. However, the group plans on completing this in the upcoming sprint.

PBI 2:

[Web Create] As a content creator, I want to be able to put general information on the new courses I'm creating - difficulty 2, value 10,000

- This PBI is also near completion but is also missing the upload file acceptance criteria as the previous PBI. The

group plans on completing this PBI in the upcoming sprint as well.

PBI 4:

[Web Create] As a content creator, I want to be able to put a lecture on the section I'm creating - difficulty 4, value 10,000

- This PBI is also near completion but is also missing the upload file acceptance criteria as the previous PBI. There are also a variety of changes that have to be made to the lecture page that were requested through the user test discussed previously.

9.5.7 Sprint 4 Retrospective

This sprint brought some unique challenges. Firstly, there was a User test and User feedback session with some of the stakeholders, which resulted in valuable insight into the usability and user experience of the current app development. This helped emphasize the importance of some unsure alignment in goals between the different PBIs. The theme of improving clarity in what the team was able to do in a sprint and informing the stakeholders from the last sprint was still prominent. The current sprint was also planned to be app development-focused and should've been a sprint where the focus in PBIs was laid towards the app developers, which it was, but due to a lack of clarity and communication in the previous sprint, which was web-oriented sprint was there still a lot to do. This resulted in a push of the deadline for the user test being pushed from sprint 3 to 4. So, the group still had a lot of work cut out for us.

What went well The group managed to shape a new foundational understanding because of the user test. The group saw a need for clarity when creating the UI for web development. Furthermore, the group was able to push a lot of changes that were not pushed in the previous sprint. And with those changes pushed and tested, it gave a sense of accomplishment, of actually seeing the group's work in use.

What did not work Generally, there was a feeling of pressure across all group members as the group was behind a deadline that was now pushed. But the group happily pushed through. There were still problems with having pull requests that were too big and contained a lot of code, but it was chopped into two pull request pushes in this sprint. Again, the push of the code was delayed due to conflicting code because of the large pull requests, but the group has managed to make this process better through experience from the last two sprints.

What has to be changed The group is a lot more fulfilled and ready to work hard, as the group now has seen the results of the hard work that has been put in. Furthermore, the group has realized more about what the individual capabilities are across the group, which in return helps us realize what goals the group is able to set.

9.6 Sprint 5

Duration of sprint 6/11/2023 - 19/11/2023

Based on the work and progress done in the previous sprint the product owner has stated in collaboration with the groups that the goal for sprint 5 is: "Deliver a stable production release for both the app and web. Implement a comprehensive points and levels system in the app. Releasable of the highest priority PBIs of offline access, video streaming, certificates, and institutional onboarding features. Address design issues on the web platform [32]."

9.6.1 Sprint 5 planning

For the fifth sprint, the group continued with the three PBIs that were not completed during the previous sprint. These three PBIs were:

PBI 1:

[Web Edit] As a content creator, I want to be able to edit the general info of the courses I haven't published yet - difficulty 3, value 10,000

- Information Box Alignment: I want the course information box to be aligned to the right and made smaller for a more balanced layout.
- Course Information Details: Remove the "Estimated Time" field from the general course information, as it's no longer needed.

- File Upload Position: Move the "Upload File" functionality to the bottom of the information box, making it more accessible and user-friendly.
- Description Character Limit: Implement a character limit for the course description field.
- File Size Limit: Add a file size limit for all uploads.
- Remove the current course creation pop-up.

Previously the first PBI's acceptance criteria was "Adapt to style guide", and now has been updated to be more descriptive.

PBI 2:

[Web Create] As a content creator, I want to be able to put general information on the new courses I'm creating - difficulty 3, value 10,000

PBI 3:

[Web Create] As a content creator, I want to be able to put a lecture on the section I'm creating - difficulty 3, value 10,000

The group also formed new PBIs in cooperation with the product owner based on the feedback from the user test in sprint 4, and the new design. The feedback from the user test is discussed in section 9.5.5. The acceptance criteria will be shown in this sprint, since the group wrote them this time. From this feedback, the following PBIs and their acceptance criteria were formed:

PBI 4:

[Web Info] As a content creator I want a guide on the section page in form of small pop-ups - difficulty 2, value 1,000

- Section page needs 3 pop-ups (text can be seen on figma)
- Forward/Backwards/Finish Buttons
- Counter of pop-ups leftover/total

PBI 5:

[Web Info] As a content creator I want a guide on the course page in form of small pop-ups - difficulty 2, value 1,000

- Course page needs 2 pop-ups (text can be seen on figma)
- Forward/Backwards/Finish Buttons
- Counter of pop-ups leftover/total

PBI 6:

[Web Info] As a content creator, I want an explanation/guide for creating a section or course to make the process more intuitive - difficulty 2, value 1,000

- Section Creation Guide: Provide a guide on how to create a section. This guide should be user-friendly and intuitive, helping content creators effectively use the platform's features.
- Course Creation Guide: Provide a guide on how to create a section. This guide should be user-friendly and intuitive, helping content creators effectively use the platform's features.

PBI 7:

[Web Info] As a content creator I want the course creation process to be more clear with step by step checklist - difficulty 3, value 1,000

- Show check mark on the completed steps
- Make it show the corresponding component ie. sections and hide the others
- Make a button to go back to the previous point in the list
- step 1: General course information
- step 2': Course section (Add section)

- step 3: Course review

PBI 8:

[Web Create/Edit] As a content creator I want a unified course creation and editing experience - difficulty 4, value 5,000

- Both the editing and the creating page should have the same design

PBI 9:

[Web Create] As a content creator, when creating a textual lecture I want to have a more comprehensive text editing experience, difficulty 4, value 1,000

- Insert images in the text

PBI 10:

[Web Create/Edit] As a content creator I want the course editing process to be streamlined and consistent- difficulty 5, value 5,000

- Replace the section edit with a dropdown menu.
- Update lecture and exercise pop-ups to match the style guide.
- Include both video and text options in lectures.
- Set a maximum video length.
- Display combined lecture and exercise counts (e.g., 2/10).
- Create a unified "Add Lecture or Exercise" button.
- Ensure the "Add Section" button aligns with the style guide.
- Add a delete button to section cards (with a trash can icon).

Therefore, the group had a total of 10 PBIs to work with during this sprint.

9.6.2 Cypress

In this sprint, the groups were introduced to cypress testing. Cypress is used for end-to-end testing and frontend testing [37]. Cypress works by finding an element in the HTML on the frontend through their ID, content, or other parameters. Then, it either reads and checks its content or interacts with it [38].

Cypress, in a way, functions a lot like an automated user. However, instead of only interacting with the element, the developer can set up checks in Cypress to ensure the HTML aligns with the design. This way, it can give feedback in the form of an error on what it expected versus what it got to indicate to the developer what has changed and needs fixing before deploying.

The previous sprints did not have many ways to test frontend development. The project technical manager decided that more testing was needed for the frontend and introduced Cypress. This was then put into the testing stack, which made it a requirement in the definition of done. Before cypress testing was introduced, only Jest testing with snapshots of the HTML code was used, as mentioned in sprint two section 9.3.4, and frontend testing was not required in the definition of done. The Jest testing had some problems with static snapshots, meaning that they had to be updated every time anything was changed in the HTML, as well as being difficult to check for functionality on the different pages.

9.6.3 Daily scrum meetings

During the sprint, the team had daily scrum meetings as in the previous sprints. The meetings gave time to reflect on and discuss the different PBIs with all team members. The meetings were generally about distributing the tasks that were made with the new design in mind. The group had many discussions about the new design to ensure a thorough understanding of it. Furthermore, the group discussed the design details with the product owner and discussed the prioritization of the PBIs.

9.6.4 Integration team

The integration team had three meetings during sprint 5, discussing the importance of dividing the code on the web into smaller components. The integration team also discussed how the group should prepare the code for the next year's students, including decreasing the technical debt and deleting unused code and files.

9.6.5 Increments

During this sprint, the group received a design made by the designer in Brasilia. The design was made after the user test with the aim of making the page more intuitive. The new design resulted in changes in the visual and functional features.

The group finally resolved the problem related to the buckets that were first encountered in sprint 2 written in section 9.3, making it possible to upload pictures and videos on a course. This means that these two PBIs were finally completed:

[Web Edit] As a content creator, I want to be able to edit the general info of the courses I haven't published yet
[Web Create] As a content creator, I want to be able to put general information on the new courses I'm creating

A pair within the group chose to start this sprint with a focus on the following PBI's:

[Web Info] As a content creator I want a guide on the section page in form of small pop-ups
[Web Info] As a content creator I want a guide on the course page in form of small pop-ups

These PBI's were made after the new design in Figma which now had tool-tips on the section page and the course page. The purpose of the tool-tips was to give the user a better understanding of the page they are on, which turned out to be necessary after the user test. An example of a text in a tool tip is: "Nossos cursos são separados em seções e você pode adicionar quantas quiser!" ("Our courses are separated into sections and you can add as many as you want!" in Portuguese).

Another PBI which was worked on during the sprint was:

[Web Info] As a content creator, I want an explanation/guide for creating a section or course (represented by the yellow attention bar in Figma) to make the process more intuitive.

The explanation/guide is a text that is permanently shown on the page. An example of this is: "Fique atento! Você pode adicionar até 10 itens em cada seção, entre aulas e exercícios." ("Pay attention! You can add up to 10 items in each section, between classes and exercises." in Portuguese). The text explains essential information about the sections.

In order for the website to follow the new design, the course creation had to be changed. Therefore the group also worked on this PBI:

[Web Edit] As a content creator, I want to be able to edit the general info of the courses I haven't published yet

The group changed the "create course" pop-up to a page, aligning it with the new design.

9.6.6 Sprint 5 Review

While the group didn't merge any code onto the dev branch in this sprint, a fair amount is written and ready to get pushed in the next sprint. Unfortunately, none of the PBIs could meet the definition of done and was therefore not accepted by the PO. Some PBIs were delayed because the group had to prioritize a task to help other groups. For the other groups to proceed with their tasks, they relied on having the lectures and exercises in the same array. Once the group had changed the implementation, the group had to rewrite some other code to align it to the modifications.

9.6.7 Sprint 5 Retrospective

In this sprint, the group had a design guide that gave them clearer directions to follow and made their work process easier when they had a clear target to follow. Having a design guide reduces the time spent considering the design, and makes it possible to immediately start coding.

What went well The group managed to fix the problem with the picture and video upload with buckets from Sprint 2, and furthermore almost finished the PBIs required to make tool-tips and explanations on the pages. Additionally, the group also made changes to the course creation page. This implies that a big portion of the code is ready to merge, requiring no further coding effort in the flow.

What did not work The integration team discussed some things about the app, that were not directly related to the group's tasks. Some of the other teams had prioritized getting a lot of functionality into the app before putting it on the Play Store, however, a more effective approach would have been to just launch the Play Store first and work on all the functionalities later. Whether the app was on the Play Store or not would not affect the group since it is not related to web development. Additionally, the integration team once again talked about the importance of making smaller pull requests only containing a single PBI each. The team should take this into account, as it has been one of the major weaknesses throughout multiple sprints.

What has to be changed The group thought that the the discussions in the integration team were generally focused about making smaller increments, which is something the group should work on improving and, therefore will focus on even more in the upcoming sprint

9.7 Sprint 6

Duration of sprint 20/11/2023 - 1/12/2023

In the final sprint to ensure a stable product version, the product owner decided that the first week should be used to finish developing PBIs, and the second week to stabilize the product/s. Thus the final sprint goal was presented by the PO: *"Complete the development and ensure the integrated launch of the Educado platform to make it accessible to all users. In the app, ensure a gamified experience and offline access. On the web, implement institutional onboarding, enhance usability, and enable efficient certification issuance for activities on Educado [32]."*

9.7.1 Sprint 6 planning

The group decided to follow the plan provided by the PO. The group decided to first and foremost get the finished PBIs merged, and then continue developing the rest of the PBIs. The group had to work with the following PBIs:

PBIs finished in the previous sprint - Merged in this sprint

PBI 1:

[Web Info] As a content creator I want a guide on the section page in form of small pop-ups - difficulty 1, value 5,000.

PBI 2:

[Web Info] As a content creator I want a guide on the course page in form of small pop-ups - difficulty 1, value 5,000.

PBI 3:

[Web Info] As a content creator, I want an explanation/guide for creating a section or course (represented by the yellow attention bar in Figma) to make the process more intuitive - difficulty 1, value 5,000.

PBI 4:

[Web Create] As a content creator, I want to be able to put general information on the new courses I'm creating - difficulty 1, value 10,000.

PBIs in development

PBI 5:

[Web Info] As a content creator I want the course creation process to be more clear with step by step checklist - difficulty 4, value 10,000.

PBI 6:

[Web Create/Edit] As a content creator I want a unified course creation and editing experience - difficulty 5, value 10,000.

PBI 7:

[Web Create] As a content creator, when creating a textual lecture I want to have a more comprehensive text editing experience - difficulty 5, value 1,000.

PBI 8:

[Web Create/Edit] As a content creator I want the course editing process to be streamlined and consistent - difficulty 4, value 10,000.

PBI 9:

[Web Create] As a content creator, I want to be able to put a lecture on the section I'm creating - difficulty 2, value 10,000.

PBI 10:

[Web Edit] As a content creator, I want to be able to edit the general info of the courses I haven't published yet - difficulty 1, value 10,000.

PBI 11:

[Collaborative Task] Make staging functional again - difficulty 3, value 10,000.

9.7.2 Daily scrum meetings

With this concluding sprint every group member will have experienced being a scrum master. As in all previous sprints, having daily meetings is a natural occurrence for the group, though the scrum master still has the responsibility to ensure the team uses scrum.

9.7.3 Integration team

As the project deadline drew near, the integration team held a total of four meetings (two each week), with the main focus shifting towards status check-ups from the individual groups. The first two meetings were about the status of PBIs, where it was decided that the strictness of the definition of done and therefore the PBIs acceptance will be lessened, and the second two meetings were about the status of deploying/resolving issues with the products. During the first two meetings, it was deemed unrealistic that all of the group's PBIs could be accomplished in one week. As a result, the product owner emphasized the value of the remaining PBIs and urged the group to continue working on the PBIs in the final week.

9.7.4 Increments

As usual, the group decided to work in pairs, and it was discussed that one pair should have the responsibility to create a merge request for the already finished PBIs, before starting on further developing other PBIs. Additionally, this pair also decided to work on *"[PBI] Make staging functional again"*.

Initially, the group thought that implementation of the website's visual overhaul wouldn't prove that big of a challenge since most of the functionality had already been implemented, and only styling had to be adapted. In contrast to their initial expectations, the implementation process proved to be much more demanding, especially due to differences in the website structure, from a multi-page website to a hybrid website.

The value of nearly every PBI is substantially high, therefore the group decided to first develop PBIs that are less dependent on one another, taking this into consideration, the following PBIs were chosen:

- *"[PBI] As a content creator I want the course creation process to be more clear with a step by step checklist"*
- *"[PBI] As a content creator I want the course editing process to be streamlined and consistent"*
- *"[PBI] As a course creator I want a unified course creation and editing experience"*

Development and implementation of these PBIs will make the website into a hybrid website, meaning the website content will be dynamically changed depending on the users' interactions, instead of redirecting to another page.

The PBI "[PBI] As a content creator I want the course creation process to be more clear with a step by step checklist", had the following criteria: have check-mark steps, and have 3 steps: general information, course sections and course review (and naturally show the corresponding component). Time was mostly spent on implementing the design from the style guide, more specifically time was spent researching tailwind documentation on properties and attributes. Implementation of functionality proved not to be that difficult because before starting development, a discussion was held on how implementation should be approached. The criteria for reaching the "Course review" checklist step is not implemented, since the PBI for the course review component was out of the scope of this sprint, though the code was written with future development in mind and can be easily implemented. The implementation of the PBI can be seen in figure 15.

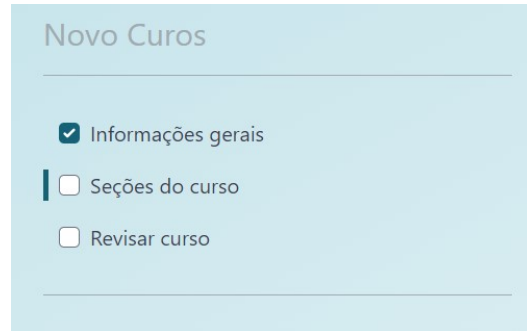
The image shows a light blue rectangular box representing a checklist interface. At the top, the text "Novo Curos" is displayed in a dark grey font. Below this, there is a horizontal line. Under the line, there are three items, each consisting of a square checkbox followed by text. The first item has a checked checkbox (with a dark blue checkmark) and the text "Informações gerais". The second item has an unchecked checkbox and the text "Seções do curso". The third item has an unchecked checkbox and the text "Revisar curso". Another horizontal line is at the bottom of the box.

Figure 15: Checklist design.

For the "[PBI] As a content creator I want the course editing process to be streamlined and consistent" the group had the following acceptance criteria: sections should be in a drop-down list, add/delete a section button matching the new style guide, be able to change the order of sections or exercises and lecture, have a unified create lecture or exercise button, adapt style of create lecture and exercise pop-ups, display combined lecture and exercise counts (e.g., 2/10). Some of these acceptance criteria had features implemented from previous sprints, which made it easier to adapt them to the new style guide.

Both the buttons for the sections and exercises/lectures were fairly simple to implement. However, some small changes in the already existing functionality were necessary, accompanied by a complete visual overhaul.

The acceptance criteria for having sections in a drop-down list was a large task and caused a lot of issues. Previously the sections were mapped to a list, this had to be further developed such that the content of the sections is also included in the mapping and the sections should be in a drop-down style. All of the section contents could be used but had to be adapted to match the new style guide, which also meant that a lot of the functionalities also had to be reworked. The functionality of creating/editing/deleting a lecture or exercise was the most problematic. After having one exercise/lecture, when trying to perform a create/edit/delete of another exercise/lecture the ID would always revert to the first instance, this was quite tricky to solve and took many resources to fix.

The order rearrangement of the sections or exercises and lectures was partially implemented by last year's students but only on sections, this was fully developed to function for the sections, and an implementation for the lectures/exercises was also developed. The remaining acceptance criteria on adapting the style of creating lecture and exercise pop-ups, and displaying combined lectures and exercise count, were not implemented.

Figure 16: Course unified design page 1.

The only acceptance criteria for the PBI "[PBI] As a course creator I want a unified course creation and editing experience", is to ensure a consistent page design for creating and editing a course. This criteria was implemented in a dynamic way, meaning the page displays content depending on the users' actions. The functionality had to be altered to differentiate if the user is creating or editing a course, and depending on the action either make a create request or update request.

There were multiple acceptance criteria for the PBI "[Web Edit] As a content creator, I want to be able to edit the general info of the courses I haven't published yet", all being about adapting the styling of the page to the new style guide. Implementing the new style was not that difficult, considering that only the already existing design had to be adapted to the new one. The implementation can be seen on figure 17

Figure 17: Course unified design page 2.

The acceptance criteria for [Web Create] As a content creator, I want to be able to put a lecture on the section I'm creating were: size limitations for video upload, display the uploaded video, and option for a textual lecture. Due to time constraints, only the feature for the textual lecture managed to be implemented, which was fairly simple and can be seen in figure 18

Crie sua nova aula

Preencha o formulário e inicie sua nova aula!

Título

Insira o título da aula

Descrição

Insira o conteúdo escrito dessa aula

Tipo de conteúdo

☐ Video ☒ Texto Estilizado

Formate o seu texto abaixo 0/800

Insira o conteúdo escrito dessa aula

Criar Cancelar

Figure 18: Textual lecture.

The remaining PBI "[PBI] As a content creator, when creating a textual lecture I want to have a more comprehensive text editing experience", was down-prioritized and wasn't developed on at all, due to having a high difficulty with a low value.

9.7.5 Sprint 6 Review

This sprint's increment was deemed successful by the product owner, all PBIs except one were implemented, though all acceptance criteria for some PBIs were not met, due to the project's deadline nearing. However this could be allowed, since having more frequent communication with the product owner, resulted in them recognizing and pointing out the most valued and essential acceptance criteria for a fulfilled PBI. This approach ensured that the product owner could achieve their goal of having a functioning and presentable product for showcase with stakeholders and users. Due to a lack of communication an issue between the group and another group, resulted in an important feature regarding image/video upload on the web to be broken, due to a change in the buckets dependency by the other group. Additionally, some of the developed features were missing some acceptance criteria, and all missing features were noted for further developers as this was the last sprint.

9.7.6 Sprint 6 Retrospective

What went well

Collaboration internally in the group functioned well, and the group had become much more open and better at communicating with the product owner, by having more transparency with the progress of the increment.

What did not work

Communication and collaboration between the group and the integration team were not the best and worsened. The group managed to have more pull requests, but the size of the pull requests was still on the larger size, leading to a more prolonged review process, therefore having a longer acceptance wait time.

What has to be changed

The group had to change from not only improving the communication processes to, more importantly, maintaining good communication, which was not the case in this sprint. Although communication became better in one place, the opposite happened in another. Therefore it is important to bear in mind not only to introduce new or improve existing practices but to also maintain them.

10 Implementation

In this section, the group will document the final state of the code developed throughout this project. The group ended up exclusively working on the website for Educado's content creators and will therefore only refer to the Frontend, and Backend parts of the product as they were the group's contribution to the project and can be found on GitHub. Furthermore, this section will only include the most interesting functions, pages, components, etc., to limit the scope and focus on key elements that are essential for a comprehensive understanding of the code developed by the group.

10.1 Front End

In this section, the code that the group has implemented in the frontend to fulfill PBIs will be discussed, described, and showcased. Firstly the group will discuss the code that is responsible for submitting newly created sections in section 10.1.1, then the code responsible for submitting the component list will be described and showcased in section 10.1.2. This is followed by a section 10.1.3 about the course creation and lastly section 10.1.4 which showcases the code that generates the tooltips that the group has created.

10.1.1 Submitting sections

For the reworked design of the section editing and creation experience, the group needed to consolidate all sections on the course such that they could be edited on the same page, but this posed the issue of saving all the sections and their components (lectures or exercises) when the user clicked save. To solve this issue the group used a modified observer pattern[39] to make all sections and their components, get notified when the user clicked save. The alterations made to the standard observer pattern were designed to accommodate the group's choice to use the functional components instead of class components, which made it impossible to register the component as an object. The group addressed this challenge by maintaining a list of references to functions within the sections. These functions are responsible for handling the saving of individual sections and their respective components.

For the implementation of the group's modified observer pattern, the group used a react state hook to store the functions that should be executed when a section is saved, which the group called "onSubmitSubscribers" as seen on line 1 of the code, which can be seen in listing 1. On line 3 to 8 the group has implemented the function "addOnSubmitSubscriber" which takes in and adds the functions that needs to be called or notified when the user submits the finished sections. The function "addOnSubmitSubscriber" works by overriding the value of the "onSubmitSubscribers" with a new array that contains the values of the previous array and the new function gotten from the input. After the "addOnSubmitSubscriber" there is the "notifyOnSubmitSubscriber" function, which is responsible for calling all the functions in "onSubmitSubscribers", this was achieved by using a for each loop, which runs all functions within the array. The last function, "onSubmit", is what the save buttons call when they are clicked. This function starts by invoking the "notifyOnSubmitSubscriber" function and then a success message is shown to the user, as seen on line 17. Lastly, the user gets a pop up asking if they want to leave the page, if the user answers "Ok" they are sent to the courses page as seen on line 19 to 25.

```

1  const [onSubmitSubscribers, setOnSubmitSubscribers] = useState<Function []>([]);
2
3  function addOnSubmitSubscriber(callback: Function) {
4      setOnSubmitSubscribers((prevSubscribers) => [
5          ...prevSubscribers,
6          callback,
7      ]);
8  }
9
10 function notifyOnSubmitSubscriber() {
11     onSubmitSubscribers.forEach((cb) => cb());
12 }
13
14 function onSubmit() {
15     notifyOnSubmitSubscriber();
16
17     toast.success("Seções salvas com sucesso!");
18
19     if(confirm("Tem certeza que deseja sair? Você perderá todas as alterações feitas.")){
20         setIsLeaving(true);
21     }
22
23     if(isLeaving){
24         window.location.href = '/courses';
25     }
26 }

```

Listing 1: Implementation of the submitting sections component.

10.1.2 Submitting the list of components

A good example of a subscriber^[39] to the onSubmit notification is the code for submitting the reorder list of components i.e. lectures and exercises. As seen in the code 2 on lines 1 to 3, a React "useEffect" hook with an empty array as its last parameter is used to ensure the "onSubmit" function is only added on the first render. To add the "onSubmit" inside of the "useEffect" the "addOnSubmitSubscriber" function is called and given an anonymous function that calls the "onSubmit" function is given as its input. The "onSubmit" works by recreating the components array found on the section, in the order of the items array. Items is an array that contains the IDs of each of the section components, where the show order is saved. The "onSubmit" works by first creating an empty array for the components called "componentArray" seen on line 6, this is then used in an anonymous function inside of the "setItems" parameters found on line 7. The main functionality of the anonymous function comes from the for loop on lines 8 to 10, which goes through each element in the items in the order they are in and pushes new objects to the "componentArray", which contains the id made by mongoDB, the value for this is gotten from a map that takes a "compId"⁸ and gives out the id. Furthermore, the object contains the "compId" gotten from the items array, and lastly, it also contains a "compType" which is found in the same manner as the id.

```

1  useEffect(() => {
2      addOnSubmitSubscriber(()=>onSubmit());
3  }, [])
4
5  function onSubmit(){
6      let componentArray:any[] = [];
7      setItems((items) => {
8          for (let i = 0; i < items.length; i++) {
9              componentArray.push({_id: idMap.get(items[i]), compId: items[i], compType: componentTypesMap.get(
10                  items[i])})
11          }
12          return(items)
13      });
14      ComponentService.setComponents(sid, componentArray);
15 }

```

Listing 2: Implementation of the submitting components list

10.1.3 Course creation

In the code below in listing 11, some selected code from the CourseComponent component can be seen. The code consists of some design choices and functionality for the input fields when creating a course. At the beginning of the code, there is a HTML form, as the purpose of the rest of the code is to collect data from the users. As seen in the class name on line 2, the layout is vertical and there is space between the children. The next class name, seen on line 3, has code for the location of the title of a course.

Then there is a label, which has the text "Nome do curso" (Course name in Portuguese) shown on the website. The input, on line 5, has to get a title from the user and it has an id called "title-field" with the type text. The defaultValue and placeholder inserts the title of the course if the id already exists, otherwise, it doesn't insert anything, as seen on line 5. In the class name on line 6, there is some styling, e.g. the outline of the field changes color when the user interacts with it. The title field is set as a required field on line 7. In line 9 the code checks for errors in the title field and if the field isn't filled, the error message "Este campo é obrigatório", which means "This field is required" will appear with red text on the website. The level and category input fields are placed beside each other on the website as seen in figure 19. They are quite similar, and therefore, the code for the category will not be explained. As seen on line 15, the level input takes up half of the width. There is a label for the level, which shows the text "Nível". In line 17, there is code for selection between the different levels and it has "Selecione o nível" (Select a level in Portuguese) as the default value. Then there is some styling similar to the styling in line 6. In line 20 the difficulty is set as required. From lines 22 to 25, the different level options "select a level", "beginner", "intermediate", and "advanced" can be seen. However, the "select a level" option is disabled, to prevent the user from choosing it. In line 27 there are error checks to ensure that a level has been chosen. In line 33 there is a div containing styling for the code until line 50. In line 34 there is code to ensure that the description label and the tooltip are placed beside each other. On line 35 the label for "Descrição" (Description in Portuguese) can be seen. On line 36, the tooltip is given a text. In this case the text is: "Dica: insira uma descrição que desperte a curiosidade e o interesse dos alunos" (Enter a description that piques students' curiosity and interest in Portuguese). "TooltipAmount" is the total amount of tooltips of the component. In this case, it is two tooltips. On line 38 there is a textarea, where the user can input a description of the course. There is a limit of 400 characters and four rows for the description. If there already exists a description, it will be placed inside the text field, otherwise, it will just be empty as seen in line 39. On line 41, the text area for the description is styled, and it is required to have a description as seen on line 42. Every time the user writes a character in the description field, the length is given to the setCharCount function. On line 45 there are an error check and an error message. The character count can be seen on line 47 and 48, this will show the current amount of characters out of 400 characters on the website.

The image shows a web form with two dropdown menus side-by-side. The first dropdown is labeled 'Nível' and has a light blue border. It currently shows 'Iniciante' with a downward arrow. A dropdown menu is open below it, showing the text 'Selecione o nível' at the top, followed by three options: 'Iniciante' (highlighted with a grey background), 'Intermediário', and 'Avançado'. The second dropdown is labeled 'Categoria' and also has a light blue border. It currently shows 'Finanças pessoais' with a downward arrow. Below these two dropdowns is a large, empty light blue rectangular area.

Figure 19: Level and category options.

```

1  {/**Field to input the title of the new course*/}
2  <form className="flex h-full flex-col justify-between space-y-4" onSubmit={handleSubmit(onSubmit)}>
3    <div className="flex flex-col space-y-2 text-left">
4      <label htmlFor='title'>Nome do curso</label> {/**Title*/}
5      <input id="title-field" type="text" defaultValue={data ? data.title : ""} placeholder={data ? data.
        title : ""}
6        className="form-field bg-secondary focus:outline-none focus:ring-2 focus:ring-primary focus:
        border-transparent"
7        {...register("title", { required: true })}
8      />
9      {errors.title && <span className='text-warning'>Este campo é obrigatório</span>} {/** This field is
        required */}
10   </div>
11
12   <div className="flex items-center gap-8 w-full mt-8">
13
14     {/**Field to select a level from a list of options*/}
15     <div className="flex flex-col w-1/2 space-y-2 text-left ">
16       <label htmlFor='level'>Nível</label> {/** Level */}
17       <select id="difficulty-field"
18         defaultValue={data ? data.difficulty : "Selecione o nível"}
19         className="bg-secondary focus:outline-none focus:ring-2 focus:ring-primary focus:border-
        transparent"
20       {...register("difficulty", { required: true })}>
21         {/**Hard coded options by PO, should be changed to get from db*/}
22         <option disabled>Selecione o nível</option>
23         <option value={1}>Iniciante</option> {/** Beginner */}
24         <option value={2}>Intermediário</option> {/** Intermediate */}
25         <option value={3}>Avançado</option> {/** Advanced */}
26       </select>
27       {errors.difficulty && <span className='text-warning'>Este campo é obrigatório</span>} {/** This
        field is required */}
28     </div>
29
30     ...{/** Category option box was here*/}
31
32   {/**Field to input the description of the course*/}
33   <div className="flex flex-col space-y-2 ">
34     <div className="flex items-center space-x-2"> {/** Container for label and icon */}
35       <label className='text-left' htmlFor='description'>Descrição </label> {/** Description */}
36       <TooltipIcon index={1} tooltipIndex={tooltipIndex} text={"😊 Dica: insira uma descrição que
        desperte a curiosidade e o interesse dos alunos"} tooltipAmount={2} callBack={setTooltipIndex}
        />
37     </div>
38     <textarea id="description-field" maxLength={400} rows={4}
39       defaultValue={data ? data.description : ""}
40       placeholder={data ? data.description : ""}
41       className="resize-none form-field focus:outline-none focus:ring-2 focus:ring-primary focus:border-
        transparent bg-secondary"
42       {...register("description", { required: true })}
43       onChange={(e) => setCharCount(e.target.value.length)}
44     />
45     {errors.description && <span className='text-warning'>Este campo é obrigatório</span>} {/** This
        field is required */}
46
47     <div className='text-right' >
48       <label htmlFor="">{charCount}/400</label>
49     </div>
50   </div>

```

Listing 3: Implementation of creating a course.

10.1.4 Tooltip

When creating a solution that could be used to solve these PBI's "[PBI] As a content creator I want a guide on the section page in form of small pop-ups", and "[PBI] As a content creator I want a guide on the course page in form of small pop-ups", the team noticed that these two have a lot of similarities and therefore it was decided that a Tooltip component should be implemented so the component could dynamically solve both PBI's when given different input. The text and the placement of the tooltips were given to the group through the Figma tool by the product owner [40].

A part of the code for the tooltip component can be seen below in the listing 4. Furthermore, some pictures of the tooltip's current design can be seen in 20, 21, and 22.

The tooltip component displays the tooltip which contains the "textContent" given. Based on the 3 variables: context given, the max number of popups in the chain and the current index number of the current popup, an if-statement decides which bottom text of the popup is most relevant. The bottom text is the bottom line which is a part of the tooltip. As an example, the bottom line which can be seen in Figure 21 Is the text: "Volta"r (...) "2/3" (...) "Próximo". The popups have a different bottom line based on whether the popup is the first, last or somewhere in the middle of the chain. To avoid repetition, not all parts of this component will be explained as much of it is similar but slightly modified to handle the visualization of different edge cases.

The component takes in four different inputs: textContent, myIndex, maxIndex, and callBack. textContent is the text that will be shown in the tooltip box as seen on line 1. myIndex is the given index in the sequence for that tooltip box. maxIndex is used to show the user how many tooltips there are in total and indicates when the sequence of tooltips will stop. This is done by checking if myIndex and maxIndex are the same. The last input is the callback, which is assigned a function in the parent class that is called when changing the index and which tooltip is shown.

The first specific piece of code worth highlighting is the textbox and the close button for the tooltip seen on lines 6 to 12. Firstly a div is used to contain both elements and to ensure their combined width is 1.75 rem, and the elements are put in a row, found on line 6. The first element is the close button, which uses a div to encapsulate the close icon to give it "onClick" functionality that forwards the index of the tooltip array beyond the max index, which hides all the tooltips on a page, the code for this can be seen on line 7. The second element is the tooltip text which is given from the component's parameters and is inserted when it is rendered.

The last part worth highlighting is one case of how the buttons on the bottom of the tooltip work, seen on line 15 to 47. To choose which button layout to show the user conditional rendering is used, of which a good example can be seen on line 31, where the condition checks if this tooltip is the first in the tooltip array, the first element of an array has the value of 0 så the condition will accept if myIndex has the value of 0. If this is the case a div is used to contain the elements and set the horizontal distance between the step counter and the next button, seen on line 32. The step count shows what step the user is on and how many there are in total, this is done inside of a p tag (paragraph tag), where the tooltips index + 1 is used to show the current step. The maxIndex, which is the length of the tooltip, is used to show the total amount of tooltips seen on line 33. The "next tooltip" button for the first tooltip in the chain says next in Portuguese. The functionality comes from the "onClick" event, which, when triggered, calls the callback function with its index increased by one as its input. This makes the next tooltip in the array be displayed, found on line 34 and how this looks on the website can be seen 20.

```

1 export const ToolTipInfoBox = ({textContent,myIndex,maxIndex, callBack}:Inputs) => {
2   return (
3     <div className="group relative ">
4       <div id={'tooltipBox'} className={`z-10 flex-col w-70 group-hover:visible
5         absolute text-white p-2 rounded-md duration-300 text-left bg-primary`} >
6         <div className='flex-col-2'>
7           <div className={`flex flex-row-reverse w-70`}>
8             <div id={'tooltipClose'} className='std-button underline
9               shadow-none hover:bg-transparent hover:text-white
10              -400 bg-transparent text-right' onClick={()=>{
11                callBack(maxIndex+1);}}>
12               <Icon path={mdiWindowClose} size={1} className="
13                 text-white " // Add cursor-pointer for hover
14                 effect
15             </div>
16             <p>{textContent}</p>
17           </div>
18         </div>
19       </div>
20       {myIndex === maxIndex-1 && myIndex === 0 ?
21         <div className='flex-col w-1/2 space-x-48'>
22           <div>{myIndex+1}/{maxIndex}</div>
23           <label id={'tooltipFinish'} className='std-button btn
24             shadow-none hover:text-white-400 underline' onClick
25             =={()=>{callBack(myIndex+1)}}>Feshar</label>
26         </div>
27         :
28         // If it is the last page, show the "Voltar/Back" "Feshar/Finish" button
29         myIndex === maxIndex-1 ?
30         <div className='flex w-1/3 space-x-40'>
31           <label id={'tooltipBack'} className='std-button
32             underline shadow-none hover:bg-transparent hover:
33             text-white-400 bg-transparent' onClick={()=>{
34               callBack(myIndex-1);}}>Voltar </label>
35           <div>{myIndex+1}/{maxIndex}</div>
36           <label id={'tooltipFinish'} className='std-button
37             underline shadow-none hover:bg-transparent hover:
38             text-white-400 bg-transparent' onClick={()=>{
39               callBack(myIndex+1)}}>Feshar</label>
40         </div>
41         :
42         // If it is the first page, show the "Próximo/Next" button
43         myIndex === 0 ?
44         <div className='flex w-1/2 space-x-40'>
45           <p>{myIndex+1}/{maxIndex}</p>
46           <label id={'tooltipNext'} className='std-button shadow-
47             none hover:bg-transparent hover:text-white-400
48             underline'onClick={()=>{callBack(myIndex+1)}}>
49             Próximo</label>
50         </div>
51         // If it is any other page, show the "Voltar/Back" "Próximo/Next" button
52         :
53         <div className='flex w-1/2 space-x-10'>
54           <div className='flex w-1/3 space-x-48'>
55             <label id={'tooltipBack'} className='std-button
56               shadow-none hover:bg-transparent hover:text-
57               white-400 underline hover:bg-transparent'
58               onClick={()=>{callBack(myIndex-1)}}>Voltar
59             </label>
60             <div>{myIndex+1}/{maxIndex}</div>
61             <label id={'tooltipNext'} className='std-button
62               shadow-none hover:bg-transparent hover:text-
63               white-400 underline hover:bg-transparent'
64               onClick={()=>{callBack(myIndex+1)}}>Próximo
65             </label>
66           </div>
67         </div>
68       </div>
69     )
70   }
71 }
72 ;}}

```

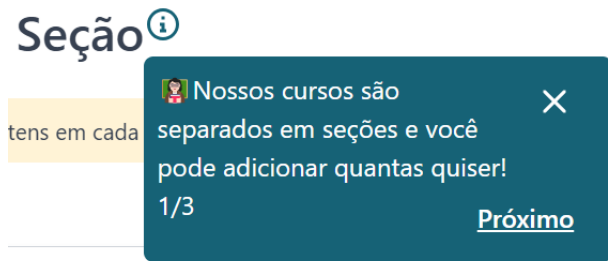


Figure 20: A presentation of how the tooltip looks.

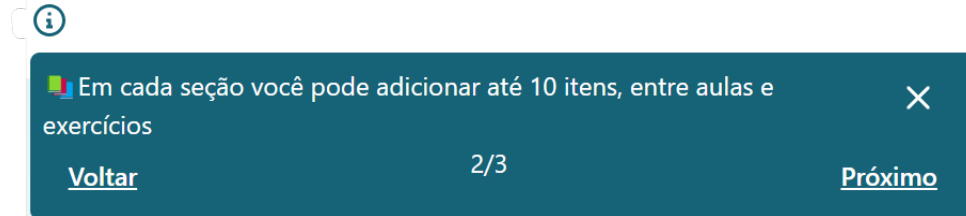


Figure 21: A presentation of how the tooltip looks.

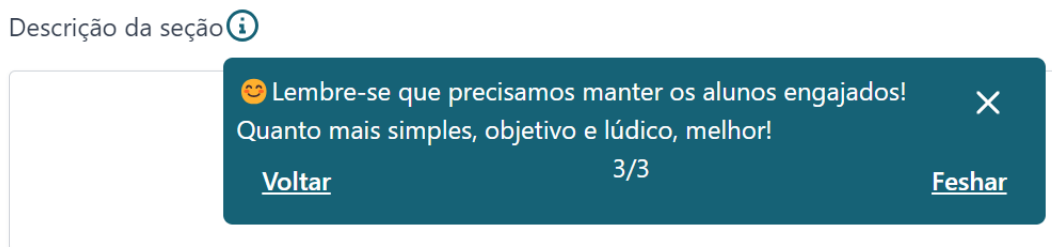


Figure 22: A presentation of how the tooltip looks.

10.2 Backend

The majority of work the group did on the back end was creating API routes for the frontend to use, each of the *get*, *put*, *patch* and *delete* routing methods of the different routes will be showcased. These routing methods are implemented in adherence with the Microsoft Azure REST API Guidelines[41].

10.2.1 Get course from id

This API route is used by the frontend and mobile application to get a course's data based on its ID. When a request for this route is made, a new asynchronous anonymous function takes in the request and response objects as its inputs seen on the first line. A try-and-catch is used in the function to handle any exception that could occur during the fetching of data. The try statement seen on lines 2 to 15, starts by getting the id from the parameters of the request found on line 3. On lines 5 to 7, it is checked if the id is a valid Mongoose Object id, if this is false a response is sent back with status 400 and an error code. After passing the first check a request to the database is made for the course with the ID given in the parameters of the request located on line 10. After the data have been received from the database, a check on line 13 ensures that the course object is set to something. If this check is not passed, a response of 404 and an error code is sent, seen on line 15. If the code passes through both checks, the course object is returned, found on line 17. If any exception occurs inside of the try statement, the catch statement sends a 500 response and an error code.

```

1 router.get('/:id', async (req, res) => {
2     try {
3         const { id } = req.params;
4
5         if (!mongoose.Types.ObjectId.isValid(id)) {
6             return res.status(400).send({ error: errorCodes['E0014'] }); // If id is not
              valid, return error
7         }
8
9         // find a course based on it's id
10        const course = await CourseModel.findById(id);
11
12        // check if courses exist
13        if (!course) {
14            // Handle "course not found" error response here
15            return res.status(404).json({ 'error': errorCodes['E0006'] });
16        }
17        res.send(course);
18    } catch (error) {
19        return res.status(500).json({ 'error': errorCodes['E0003'] });
20    }
21 });

```

Listing 5: Implementation of the get course API route.

10.2.2 Creating a new lecture

To add a new lecture to the database, the put request was used with the parent section ID of the lecture to be added as the parameter. Lines 2 to 4, get the values that should be used for the new lecture and assign the values to variables, which is followed by a check of the value of the content to ensure that it is not null or undefined, but if the values are null or undefined it is overwritten and set to an empty string, this check is a temporary precaution as some content types do not upload correctly at the time of writing this. When data from the request has been processed and assigned variables, a new "LectureModel" object is made with the values from the variables which can be found on lines 11 to 19. After the object has been created a *try statement* contains the code for saving, it starts by getting the parent section from the database which is then used to check if the section already has the maximum amount of components allowed. If it does a response with error status 400 and code "E1101" as seen on lines 22 to 26. Following the check of the data from the "LectureModel". The object is saved to the database linked to the parent sections component array as an object with the lecture ID and its type, i.e., the lecture is added, and saved in the database. If no exceptions have occurred, the created lecture data is sent in the response with the status of 201. If an exception happens the catch statement sends a response with the status of 400 and an error code of "E0000".

```

1 router.put('/:section_id', /*requireLogin,*/ async (req, res) => {
2     const {title, description, contentType} = req.body; //Handles the data in "data" from the
      request
3     const section_id = req.params.section_id; //Handles the data in "params" from the request
4     let {content} = req.body;
5
6     //Video upload not fully implmented yet, therefore if its video we set it to an empty string
7     if (content === null || content === undefined){
8         content = '';
9     }
10
11     const lecture = new LectureModel ({
12         parentSection: section_id,
13         title: title,
14         description: description,
15         contentType: contentType,
16         content: content,
17         dateCreated: Date.now(),
18         dateUpdated: Date.now()
19     });
20
21     try {
22         const section = await SectionModel.findById(section_id);
23
24         if(section.components.length >= 10){
25             res.status(400).send({error: errorCodes['E1101']});
26         }
27         await lecture.save();
28         await section.components.push({compId: lecture._id, compType: 'lecture'});
29         await section.save();
30         res.status(201).send(lecture);
31     } catch (err) {
32         res.status(400).send({error: errorCodes['E0000']});
33     }
34 });

```

Listing 6: Implementation of the put Lecture API route.

10.2.3 Updating a section components

When the update section route is called which can be seen in listing 8, an asynchronous function is created to handle the request as seen on line 1. This is followed by getting the section ID from the request's parameter and getting the components from the requested body on lines 2 and 3. Then a for loop is used to translate the IDs from the components array into MongoDB ObjectIds, the logic for doing this is, first an individual component is gotten from the array, followed by overwriting its “_id”, which is the id for the object, with an equivalent MongoDB ObjectId. It then does the same for the “compId”, which is the id for the lecture or exercise, the loop and logic can be seen on lines 5 to 10. After all IDs, the components have been translated the section that should be updated is found in the database and assigned to a variable, which is used to call the updateOne method with the \$set attribute and the components, this overwrites the value in the components array in the section with the one sent in the request, seen on 12 and 14. A catch statement is used after the updateOne method to handle any exceptions that might occur, is found on lines 15 to 17. Lastly, the updated section is sent as a response seen on line 18.

```
1 router.patch('/:sectionId', async (req, res) => {
2     const { sectionId } = req.params;
3     const { components } = req.body;
4
5     for (let i = 0; i < components.length; i++) {
6         const comp = components[i];
7
8         comp._id = mongoose.Types.ObjectId(comp._id);
9         comp.compId = mongoose.Types.ObjectId(comp.compId);
10    }
11
12    const section = await SectionModel.findById(sectionId);
13
14    section.updateOne({$set:{components : components}})
15    .catch(err => {
16        console.log(err);
17    });
18    res.send(section);
19 });
```

Listing 7: Implementation of the patch Components API route.

10.2.4 Deleting a section

The delete section route requires the id of the section that should be deleted as its parameter, to handle the request an asynchronous function is used with the request and response as its inputs, as seen on line 1. The function starts by getting the id from the request parameter, the id is then used to get the section from the database. If the section can not be found in the database a response with status 204 is sent indicating that the object is not in the database, as the request wanted, this can be seen on lines 2 to 7. When the section is found, it is first removed from its parent course *sections* array by calling the `CourseModel.updateOne`, the section's *parentCourse value* is used to filter by id and the `$pull` updateQuery is used in the sections array with the section's id as a filter, seen on line 10. Afterward, the sections' components are saved in a variable, a for loop is then used to go through each component in the array and check their type i.e. lecture, or exercise, this is used to choose the model to delete from, but in both cases, the “`findByIdAndDelete`” method is used with the id as input, seen on line 10 to 22. Lastly, the section itself is deleted using the “`deleteOne`” method in the `SectionModel` with a filter on the id, and again if the section cannot be found a response with status 204 is sent, if no exceptions are accorded a response with status 200 is sent, seen on line 25 to 28.

```

1 router.delete('/:id'/*, requireLogin*/, async (req, res) => {
2     const { id } = req.params;
3
4     // Get the section object
5     const section = await SectionModel.findById(id).catch((err) => {
6         res.status(204).send(err);
7     });
8
9
10    // Remove the section from the course section array
11    await CourseModel.updateOne({_id: section.parentCourse}, {$pull: {sections: section._id}});
12
13    // Get lecture array from section
14    const components = section.components;
15
16    // Delete all lectures and excercises in the section
17    for (let comp of components) {
18        if (comp.compType === ComponentType.LECTURE) {
19            await LectureModel.findByIdAndDelete(comp.compId);
20        } else if (comp.compType === ComponentType.EXERCISE) {
21            await ExerciseModel.findByIdAndDelete(comp.compId);
22        }
23    }
24
25    // Delete the section
26    await SectionModel.deleteOne({ _id: id }).catch((err) => res.status(204).send(err));
27
28    // Send response
29    res.status(200).send('Section Deleted');
30 }

```

Listing 8: Implementation of the pacht Components API route.

10.3 Test

The following section consists of different tests that were chosen and highlighted by the group for this report. There were quite a lot of tests made throughout the project, which would be infeasible to showcase and highlight every one of them.

This section will be divided into the different repositories the group worked with, i.e. frontend and back, which each had their own framework for testing. The Cypress framework was responsible for testing the frontend and the Jest framework responsible for the backend.

10.3.1 Testing through Cypress

At one point, as mentioned in section 9.6.2, the group started testing using Cypress on frontend. In this section, the group will showcase some of the testing implementations of Cypress and how it functions.

Because of how Cypress works, it will create a virtual machine that runs the code and navigates the website. This means that Cypress can not use the cookies or settings from the developer's browser. Because of this, the virtual machine has to navigate through all the required pages to reach the page that should be tested.

In the case of Educado, the login page to create the access token is the only required page. This means that the group had to set up a login sequence that would be run in every test.

When running the before function in Listing 9, it tells Cypress that whatever happens in the function must be run before any test. This ensured the tests were run under the right circumstances and environment. In this case, it intercepts any login attempts by Cypress and sends a login access token back. By intercepting the login, do the code bypass needing a specific by Cypress. Instead, it will use a dummy login that will be granted with any login information Cypress sends and still get access to the website.

This login sequence runs almost every test made in Cypress on the frontend. A few tests that tested the login and account creation didn't require it, but as another group made those tests, they will not be highlighted here in this report.

The group will examine how various commands interact with the HTML to delve deeper into the Cypress test. As mentioned earlier, the 'before' function tells Cypress to run the subsequent code before any tests are executed. Then, line 2 establishes an interception for the login request. This interception checks whether any POST request to the backend URL (BACKEND_URL/api/auth/login, where BACKEND_URL is a variable representing the backend URL) is transmitted. If a request meets these criteria, a response is generated with a status code of 202, including user information and a mock access token.

Moving forward, on line 14, Cypress navigates to the login page, while lines 15 to 17 involve locating and populating different fields within the login form and submitting the login request. This process uses the 'get' function to identify the HTML element with an ID matching the respective input, then utilizes the 'type' method to interact with the element and input text into it. The 'click' method is also employed on line 17 to simulate a button click.

The last things Cypress does in this sequence are checking if the received URL matches expectations and saving local storage if it does. On line 18, Cypress uses its function to check the frontends functionality and/or design, using the 'url' function to get the current URL and the 'should' function with the 'include' tag to verify it with the expected URL.

```

1 before(() => {
2   cy.intercept('POST', `${BACKEND_URL}/api/auth/login`, {
3     statusCode: 202,
4     body: {
5       userInfo: {
6         id: '1',
7         name: 'Test User',
8         email: 'test@email.com'
9       },
10      token: 'testToken'
11    },
12  });
13
14  cy.visit('http://localhost:3000/login')
15  cy.get('#email-field').type('test@email.com')
16  cy.get('#password-field').type('password')
17  cy.get('#submit-login-button').click()
18  cy.url().should('include', '/courses')
19  cy.saveLocalStorage();
20 });

```

Listing 9: Cypress login sequence.

The code in Listing 10 focuses on testing the tooltip functionality on a course page. The description block at the beginning of the code specifies the purpose of the test suite. It tests the behavior of tooltips and is organized into three individual tests.

The beforeEach block on line 8, similar to the one in the previous listing, sets up the test environment. Cypress intercepts a GET request to a specific course endpoint and provides a mock response containing course details. This ensures that the test is executed under controlled conditions.

The first test, 'Close tooltip' found on line 35, finds and clicks on the first tooltip icon on the course page. Hovering over the icon is enough to activate it, though Cypress does not have a hover function and will automatically hover the icon if it clicks it. Cypress then checks if the tooltip box exists, which ensures the functionality of the icon as well as checks the design matches the expectation. Lastly, it clicks on the close button and verifies that the tooltip box no longer exists.

The second test, 'Cycle through tooltips' found on line 44, initiates a tooltip by clicking on the first icon again, checking the tooltip box's existence, and then clicking on the next and back buttons to check their functionality. Finally, it closes the tooltip and ensures its disappearance.

The third test, 'Finish tooltip' found on line 56, clicks on the second, and in this case, the last, tooltip icon, checks the tooltip box existence, clicks on the finish button to check its functionality, and confirms that the tooltip box is no longer present.

These tests collectively assess the functionality and user interactions with tooltips on the course page, assuring the tooltips' functionality and design.

```

1  /**
2  * Test the tooltips on the course page
3  */
4  describe('Tooltip on Course Page', () => {
5
6    //... the login code from Listing: 9
7
8    beforeEach(() => {
9      cy.intercept('GET', `${BACKEND_URL}/api/courses/*`, {
10        statusCode: 200,
11        body: {
12          _id: 1,
13          title: "Test Course",
14          category: "sewing",
15          coverImg: "",
16          description: "This is a test course.",
17          creator: "Test User",
18          status: "published",
19          estimatedHours: 0,
20          rating: 1,
21          difficulty: 1,
22          numOfSubscriptions: 0,
23          dateUpdated: Date.now(),
24          dateCreated: Date.now(),
25          sections: [],
26          __v: 2
27        },
28      });
29
30      cy.restoreLocalStorage();
31      cy.visit('http://localhost:3000/courses/manager/1/0')
32    });
33
34
35    it('Close tooltip', () => {
36      cy.get('#tooltipIcon0').click()
37      cy.get('#tooltipBox').should('exist')
38
39      cy.get('#tooltipClose').click()
40      cy.get('#tooltipBox').should('not.exist')
41    });
42
43
44    it('Cycle through tooltips', () => {
45      cy.get('#tooltipIcon0').click()
46      cy.get('#tooltipBox').should('exist')
47
48      cy.get('#tooltipNext').click()
49      cy.get('#tooltipBack').click()
50
51      cy.get('#tooltipClose').click()
52      cy.get('#tooltipBox').should('not.exist')
53    });
54
55
56    it('Finish tooltip', () => {
57      cy.get('#tooltipIcon1').click()
58      cy.get('#tooltipBox').should('exist')
59
60      cy.get('#tooltipFinish').click()
61      cy.get('#tooltipBox').should('not.exist')
62    });
63
64  });

```

Listing 10: Implementation of tooltips on a course using Cypress.

The code in Listing 11 is dedicated to testing the functionality of creating a new course on the course manage page. The test suite is named 'Create Course', and it begins by setting up the necessary environment using the `beforeEach` block on line 8, including the login code from a previous listing. Cypress intercepts a PUT request to the backend API endpoint responsible for creating courses and responds with a mock status of 200 and a body indicating the status of the created course as "draft".

The primary test within this suite, 'Create Course' found on line 20, focuses on the process of creating a course. It starts by checking if the title and description fields are initially empty, lines 23 and 24. Subsequently, on lines 27 to 30, it fills in various fields such as title, description, category, and difficulty using Cypress commands like `type` and `select`.

The test then checks if the fields have been correctly populated by asserting their values on lines 33 to 36. Finally, it triggers the save action by clicking the button with the element ID 'SaveAsDraft'. This comprehensive test ensures that the course creation functionality works as intended, covering field population, field value assertions, and the course creation activation.

```

1  /**
2   * Test the create course page
3   */
4  describe('Create Course', () => {
5
6     //... the login code from Listing: 9
7
8     beforeEach(() => {
9         cy.intercept('PUT', `${BACKEND_URL}/api/courses/`, {
10             statusCode: 200,
11             body: {
12                 status: "draft"
13             },
14         });
15
16         cy.restoreLocalStorage();
17         cy.visit(`http://localhost:3000/courses/manager/0/0`);
18     });
19
20     it('Create Course', () => {
21
22         //check if title and description field is empty
23         cy.get('#title-field').should('be.empty')
24         cy.get('#description-field').should('be.empty')
25
26         //fill in the different fields
27         cy.get('#title-field').type('Test Course')
28         cy.get('#description-field').type('This is a test course.')
29         cy.get('#category-field').select('sewing')
30         cy.get('#difficulty-field').select('1')
31
32         //check if the fields are filled in
33         cy.get('#title-field').should('have.value', 'Test Course')
34         cy.get('#description-field').should('have.value', 'This is a test course.')
35         cy.get('#category-field').should('have.value', 'sewing')
36         cy.get('#difficulty-field').should('have.value', '1')
37
38         //save the course
39         cy.get('#SaveAsDraft').click()
40
41     });
42
43 });

```

Listing 11: Implementation of creating a course using Cypress.

10.3.2 Backend testing

A lot of backend testing was done throughout the project, as the tools and test stack were implemented for the backend from the start. The group has selected a few tests from the exercise route to highlight, as they reflect well how the test across the different routes works. A lot of the backend tests the group created for one route were

similar to the other routes the group worked on, as they all had much the same purpose but each with a unique data element.

All of the test for the backend repository was created with the Jest framework for React. As mentioned above, this framework was already in place when the group started working on the project.

Listing 12 is the implementation of a test for creating an exercise in Educado's backend. This test is designed to ensure that the backend service correctly handles the creation of new exercises for specific course sections.

The test environment is initially set up using a `beforeAll` function on line 5. In this setup, a connection to a temporary database is established on line 6, and necessary fake data, such as a user and a section, are inserted into the database on lines 9 and 10. This preparation ensures that the test has all the required data and the right environment for execution.

The core test, on line 13 titled 'Creates an exercise for a given course', is an asynchronous function, which reflects the nature of backend operations. It starts by generating an access token for the fake user, which is necessary for authenticating the API request. Then on lines 15 to 18, the test makes an HTTP PUT request to the backend endpoint (`/api/exercises/ID`, where ID is the ID of the fake section), sending along the necessary exercise data (like title, question, and answers) and the generated access token for authorization. The test expects a successful response on line 19 with a status code of 201, which indicates that the exercise has been created successfully. The response body is further checked on line 21 to see if the exercise's title matches the expected value. This step verifies that not only the exercise is created, but it is also created with the correct details.

Finally, on line 24, the `afterAll` function is used to clean up the test environment. It deletes all documents in the 'users', 'sections', and 'exercises' collections of the database on lines 25 to 27. This step is important to ensure that the test environment is reset and does not affect subsequent tests.

```

1 describe('Create exercise route', () => {
2
3   let db; // Store the database connection
4
5   beforeAll(async () => {
6     db = await connectDb(); // Connect to the database
7
8     // Insert the fake user into the database
9     await db.collection('users').insertOne(fakeUser);
10    await db.collection('sections').insertOne(fakeSection);
11  });
12
13  it('Creates an exercise for a given course', async () => {
14    const token = signAccessToken({id: fakeUser._id});
15    const response = await request(app)
16      .put('/api/exercises/' + fakeSection._id)
17      .set('Authorization', `Bearer ${token}`)
18      .send({ title: 'Test exercise', question: 'Test question', answers: {text: "test", correct: false,
19        feedback: "test"} })
20      .expect(201);
21    expect(response.body.title).toBe('Test exercise');
22  });
23
24  afterAll(async () => {
25    await db.collection('users').deleteMany({}); // Delete all documents in the 'users' collection
26    await db.collection('sections').deleteMany({}); // Delete all documents in the 'sections' collection
27    await db.collection('exercises').deleteMany({}); // Delete all documents in the 'exercises'
28      collection
29  });
30 });

```

Listing 12: Implementation of test, that test the creating an exercise in the backend.

Listing 13 describes a test suite for the 'PATCH: Update exercise route' in the backend of Educado. This test aims to verify that the backend correctly handles the updating of existing exercises.

The test environment is set up very similar to the previous Listing 12 using the `beforeAll` function at the start on line 5. This setup involves connecting to a temporary database on line 6 and inserting necessary mock data, such as a user and an exercise on lines 9 and 10, into the database. This preparation is essential to simulate an environment where an exercise already exists and needs updating.

The main test, on line 13 titled 'Update the fake exercise', is asynchronous, much like the Listing 12, to work with the HTTP requests. It begins identically to Listing 12 by generating an access token for the fake user on line 14, which is necessary for authenticating the API request. Next, the test executes a PATCH request to the backend endpoint (`/api/exercises/ID`, where ID is the fake exercise's ID) on lines 15 to 17. This request includes the updated exercise data, such as a new title and question, and the access token for authorization. The test expects a successful response with a status code of 200 on line 19, indicating the exercise has been updated successfully. To verify the update, the response body is checked on lines 21 and 22 to ensure the exercise's title and question match the updated values. This step confirms the that exercise was updated and with the correct details.

Finally, the `afterAll` function, starting at line 25, cleans up the test environment. It deletes all documents, on lines 26 and 27, in the 'users' and 'exercises' collections, ensuring the test environment is reset and does not interfere with subsequent tests.

```

1 describe('PATCH: Update exercise route', () => {
2
3   let db; // Store the database connection
4
5   beforeAll(async () => {
6     db = await connectDb(); // Connect to the database
7
8     // Insert the fake user into the database
9     await db.collection('users').insertOne(fakeUser);
10    await db.collection('exercises').insertOne(fakeExercise);
11  });
12
13  it('Update the fake exercise', async () => {
14    const token = signAccessToken({id: fakeUser._id});
15    const response = await request(app)
16      .patch('/api/exercises/' + fakeExercise._id)
17      .set('Authorization', `Bearer ${token}`)
18      .send({ title: 'Test', question: 'Test question' })
19      .expect(200);
20
21    expect(response.body.title).toBe('Test');
22    expect(response.body.question).toBe('Test question');
23  });
24
25  afterAll(async () => {
26    await db.collection('users').deleteMany({}); // Delete all documents in the 'users' collection
27    await db.collection('exercises').deleteMany({}); // Delete all documents in the 'exercises'
28    collection
29  });
30  });

```

Listing 13: Implementation of test, that test updating an existing exercise in the backend.

Listing 14 shows the implementation of the test for the 'DELETE: Delete exercise route' in the backend of Educado. This test is focused on ensuring that the system correctly handles the deletion of existing exercises.

The setup for this test suite begins with a chunk of code, from lines 6 to 23, that is almost identical to the Listing 12. However, in this instance, the code not only sets up the environment by connecting to a temporary database and inserting mock data, such as a user and a section but also saves the response of the exercise creation in a variable named `exerciseCreate`. This approach provides a direct reference to the newly created exercise, which is essential for the deletion test.

The primary test, on line 26 titled 'Delete the created exercise', is an asynchronous function. It begins by generating an access token on line 27 for the fake user for authentication purposes. Following this, the test sends a DELETE request on lines 29 to 31 to the backend endpoint (`/api/exercises/ID`, where ID is the ID of the created exercise). The test expects a successful response with a status code of 200, indicating that the exercise has been deleted. The

response text is then checked on line 34 to ensure it matches the expected confirmation message "Exercise Deleted". This verification confirms the backend's capability to delete exercises.

Finally, the `afterAll` function, starting on line 37, is used to clean up the test environment. It involves deleting all documents in the 'users', 'sections', and 'exercises' collections of the database. This step resets the test environment, ensuring it does not impact subsequent tests.

```

1 describe('DELETE: Delete exercise route', () => {
2
3   let db; // Store the database connection
4   let exerciseCreate; // Store the exercise created
5
6   beforeAll(async () => {
7     db = await connectDb(); // Connect to the database
8
9     // Insert the fake user into the database
10    await db.collection('users').insertOne(fakeUser);
11    await db.collection('sections').insertOne(fakeSection);
12  });
13
14  it('Creates a exercise for a given course', async () => {
15    const token = signAccessToken({id: fakeUser._id});
16    exerciseCreate = await request(app)
17      .put('/api/exercises/' + fakeSection._id)
18      .set('Authorization', `Bearer ${token}`)
19      .send({ title: 'Test exercise', question: 'Test question', answers: {text: "test", correct: false,
20        feedback: "test"} })
21      .expect(201);
22
23    expect(exerciseCreate.body.title).toBe('Test exercise');
24  });
25
26  it('Delete the created exercise', async () => {
27    const token = signAccessToken({id: fakeUser._id});
28
29    const response = await request(app)
30      .delete('/api/exercises/' + exerciseCreate.body._id)
31      .set('Authorization', `Bearer ${token}`)
32      .expect(200);
33
34    expect(response.text).toBe("Exercise Deleted");
35  });
36
37  afterAll(async () => {
38    await db.collection('users').deleteMany({}); // Delete all documents in the 'users' collection
39    await db.collection('sections').deleteMany({}); // Delete all documents in the 'sections' collection
40    await db.collection('exercises').deleteMany({}); // Delete all documents in the 'exercises'
41      collection
42  });
43 });

```

Listing 14: Implementation of test, that test the deletion of an existing exercise in the backend.

11 Final Retrospective

11.1 Sprint reviews and sprint retrospectives

In this project, the sprint reviews for the entire team were led by the integration team. This, however, turned out to be somewhat of a challenge since no one on the team had had any experience with conducting sprint reviews before this semester. As a result, the first sprint reviews were not properly documented and the sprint meetings were rather disorganized. However, the organization improved and got a better grasp on the structure of sprint reviews after the third sprint, which caused the subsequent sprint reviews and meetings to become increasingly well-planned and effective.

All documentation created by the integration team, such as meeting summaries, sprint review presentations, and retrospective presentations, is used as a baseline. Since the first three sprints were not documented properly by the integration team, it is difficult to reflect upon their results. However, at the project's Miro, a tool used to organize the project and keep track of backlogs and PBIs, one can find an overview of the sprints. The integration team's documentation together with the information at Miro, will be used to reflect and discuss each sprint.

11.1.1 Sprint 1

Sprint Goal: Stabilize and enhance existing features of the Educado platform, focusing on design improvements, usability fixes, and performance optimization. Prepare the foundation for future feature integration while ensuring a robust and user-friendly experience for both content creators and learners.

In the first sprint, the teams took on an average of 5-6 PBIs. It took more time than expected for all the teams to understand and stabilize the existing code base. Therefore, the sprint resulted in two accepted PBIs out of a total of 29 distributed PBIs, which is a clear indication of how much of a challenge the first sprint was and how apparent the lack of experience was.

The sprint goal for sprint 1 was not achieved.

Integration team's retrospective:

- Communicate using Discord and GitHub issues (dependencies). Everyone needs to use Discord and GitHub every time they have an issue that involves other than just their team.
- Do not worry about making the backend, frontend, and database work together immediately. Get one to work with dummy data, and tie them together when both are ready.
- If there are any questions about the product, just book a meeting with the PO. A meeting too much is a lot better than one too little.
- If there ever is an issue involving every team, the group room, or a situation where the opinions of the different teams are relevant. Book an integration meeting. Everyone can book or suggest a meeting.
- GitHub actions (CI/CD) need to be done in-depth, to get the automation on release and testing. Should probably be its own PBI.
- Sprint reviews should be shown from the staging branch, so always merge when done, and show only what is done.
- Divide PBI and work into smaller sub-PBIs, with exact acceptance criteria.
- More transparency in task distribution and progression in individual teams

11.1.2 Sprint 2

Sprint Goal: Stabilize and enhance the Educado platform by enabling web course creation and mobile app access. Focus on design, usability, and performance improvements while ensuring future feature integration and user-friendliness.

In the second sprint, the PBIs from the first sprint were transferred to the current sprint backlog. Some teams also took on a couple of extra PBIs. This sprint resulted in 9 accepted PBIs out of a total of 40 distributed PBIs, which indicates that the teams still struggled with completing the PBIs.

The sprint goal for sprint 2 was partly achieved. The mobile app and web were now stable running on localhost.

Integration team retrospective: Continue working on the findings from the last sprint review. Especially focusing on GitHub issues and cross-team functionality.

11.1.3 Sprint 3

Sprint Goal: Enhance the user experience on the web and app, including optimizing course editing on the web and improving the social-gamified learning experience on the app. Prepare for stakeholder validation and ensure deployment readiness.

In the third sprint, there were 52 PBIs in total, and 15 of them were approved. All the remaining PBIs from sprint 2 were transferred to the current sprint backlog. Even though there were only 15 of 52 PBIs approved, a large amount of PBIs only had to be tested and/or merged into the staging branch before they were considered "approved".

The sprint goal for sprint 3 was not achieved, as most PBIs were not merged to staging yet.

Integration team retrospective:

- Focus on having a stable staging branch and being sure all PBIs meet the DoD and are merged before starting on new PBIs.
- Working on stakeholders' concerns, by having a stable product to show the stakeholders.
- Better communication/cross-functionality, being more transparent and continuing to communicate with other groups about dependencies (using GitHub issues).
- Having more realistic goals and deadlines
- Working on more automation and checking if CodeScene can be integrated into GitHub actions
- Using e2e (cypress) for frontend, using jest unit tests for backend and utility functions.
- Start creating a concrete plan in the sprint retrospectives.

11.1.4 Sprint 4

Sprint Goal: Enhance the user experience on the web and app, including optimizing course editing on the web and improving the social-gamified learning experience on the app. Prepare for stakeholder validation and ensure deployment readiness.

In the fourth sprint, there were 65 PBIs, and 36 of them were approved. The sprint goal was partly achieved for sprint 4. The app and the web were built and tested with waste pickers. Gamified experience is improving and courses can be created and edited on the web.

A few features, such as the progress bar, and the navigation of lectures and exercises, are missing, which is why the sprint goal was only partly achieved.

Integration team retrospective:

11.1.5 Sprint 5

Sprint Goal: Deliver a stable production release for both the app and the web. Implement a comprehensive points and levels system in the app. Releasable of the highest priority PBIs of offline access, video streaming, certificates, and institutional onboarding features. Address design issues on the web platform.

In the fifth sprint, there were 60 PBIs, where 19 were accepted. This was the first sprint where one could build the app and deploy it for internal testing on the Play Store. Therefore, one could start to test the app on real phones. Also, the public web URL was up and running, but the web groups had some struggles migrating their PBIs from localhost to the actual web. The sprint goal for sprint 5 was partly achieved.

Integration team retrospective:

What went wrong?

By now, we should have built the app on Google Play Store, cause the first APK file had been built. We wanted to have more functionality before deploying. However, we can now see that this is a “mistake”, in terms of following agile development principles. We should instead just have deployed the app, even with functionality still missing, cause we had promised the PO, that it would be ready for downloading and testing by users. Something else that went wrong and set other groups back was that some groups created PRs with multiple PBIs, and therefore, “done” PBIs were blocked by some unfinished ones. We did not test that the features merged in staging could be built before deploying it to the server. That of course was an issue, because it ended up causing trouble when displaying the product to the PO.

How could we improve?

- Deploy to Play Store the second we have a stable APK file
- Remember to try having one PR per PBI (unless they are very small)
- NEVER merge directly into staging (even hotfixes should be in both dev and staging!)
- CI/CD pipeline from dev to staging, and from staging to main.
- This way, the app will be built for all staging merges and detect problems before production.
- The pipeline from staging to main is the only place where the app would be deployed.
- Plan is to add automated tests in this pipeline, such as e2e testing using Detox (but this can only be run after the app is built)

Sprint plan for next sprint: For the next and final sprint in which we are going to make further development on the Educado project, we will distinguish heavily between the first and second week of the sprint.

First week - Full focus on development (and deploying app + get website stable)

The first week is about finishing the last PBIs across the development teams and making the very finishing touches, to make sure we do not have any unfinished code that we push into the main and leave for the next software students, as well as to make sure that we deliver as good a product that we can, with as much functionality as possible.

Second week - Fix tests, reduce technical debt, remove unused code and files, fix README and documentation.

The second week is about polishing up the very last tests and related code, as well as comments. Comments in the code should only remain, if they are useful for the next team of developers next year, otherwise, they should be deleted. This week, documentation must also be in order. That goes particularly for the readme files, which every group, having contributed to the separate repositories, must make sure to make contributions to with their knowledge of the code they have written. Screenshots are very welcome. Finally, unused code that either is commented out or completely not used must be deleted as well, and maybe mentioned in the readme or feature markdown file in the GitHub, for the next team of students to read and familiarize themselves with, so that they can continue working on the project and implement the unfinished thoughts that we had.

11.1.6 Sprint 6

Sprint Goal: Complete the development and ensure the integrated launch of the Educado platform to make it accessible to all users. In the app, ensure a gamified experience and offline access. On the web, implement institutional onboarding, enhance usability, and enable efficient certification issuance for activities on Educado.

In the sixth and last sprint, there was only one new PBI, which resulted in a total of 42 PBIs, where 36 of them were accepted. Another important note about this sprint is that it is the first sprint where the CI/CD pipeline for the app was fully implemented. This way, all new features merged to the main, will automatically build and be released on Play Store. This was also the first sprint where all the accepted web PBIs were running on the stable website and not on localhost. The sprint goal for sprint 6 was partly achieved.

Integration team retrospective: For sprint 6 the biggest challenge was communication between the teams, most importantly between the web and app teams. The lack of good and frequent communication between the teams working on web and app development, respectively, resulted in them working with different versions of the backend. The video streaming group had created a microservice to handle videos and images and thus moved that functionality from the common backend. However, at the same time, the onboarding group created a similar but

not identical rework in the common backend, which virtually resulted in the web and app working differently with videos and images (or not at all in the web app's case). The poor communication continued to the sprint review, where these features were presented as "done" even though they did not work properly, which was because of rework of the code that then caused dependency problems, that were not identified before or after the review. Overall though, the teams were generally more flexible in terms of pull request reviews, where "features" were prioritized over "perfect code", which increased our technical debt, but also increased the functionality of the product, that this semester of software students hand over to the next.

11.2 Progress

The six sprints we had to work on this project resulted in 118 completed PBIs. Many of these PBIs were intended to stabilize the already-existing codebase and make it work. This work took longer than first anticipated, which might be due to technical abilities or misinterpreted inter-team dependencies, and as such, excessive time was spent refactoring the existing code. This mass refactoring was largely due to a lack of documentation and insufficient presentation of the existing system. This ultimately meant that we were not able to understand the full scope of the system, how to set it up, etc. This refactoring of the codebase meant that we made good progress with implementing essential features for a mobile app and web page. However, this time spent on the essential parts of the product meant that the elected 'main features' of some groups were not as highly prioritized, mainly the certificate issuance system, institutional onboarding, and virtual tutor. This issue could have been mitigated had there been sufficient documentation present from the previous team. This led us to prioritize documentation when handing over the project, as we know firsthand that insufficient documentation can see a large decrease in overall progress.

During sprints, we have identified some hindrances that have slowed our overall progress and decreased team progress velocity. Something that we as a team of teams generally struggled with, and hindered progression was often not being able to find a balance and compromise between code quality and velocity. This created some conflicts, which needed time to be resolved. Early project progress was heavily impacted by the collectively agreed upon DoD. We had branch structure challenges, where a significant challenge emerged from a lack of clarity regarding the branches' differing purposes. Violations of explicitly defined procedures and instructions happened, which impacted the whole project. Examples of this include direct commits to the staging branch, and pushing unaccepted PBIs to the main branch. Some positive aspects of the velocity of the sprints, and increasing overall progression include having designs for most of the product. This meant we had a blueprint, instead of needing to create designs ourselves. The individual progression as developers overall also went well. This includes more technical aspects, such as working with mobile applications. It, however, also includes working as scrum teams and communicating with each other. This saw a large improvement throughout the project. We find this especially important, as we prioritize learning since we are university students after all.

A critical remark to make is the continuous progress the different teams have accomplished. This metric has been tracked by filling out an agile survey that measured our assessment according to subjects. Through each sprint review, the metrics revealed that the team's effectiveness increased; after the first sprint review, the team scored 56 effectiveness. As part of the last sprint review, the subsequent score reached 67. That's almost a 20% increase in team efficiency in the relatively constrained time the team has had to build this project and comprehend the agile approach to software development.

11.3 Our Use Of The Nexus Scaling Framework

11.3.1 Nexus within the project

In practice, each member brought their ideas, opinions, and feedback on behalf of their scrum team to the integration meetings with the intention of every team influencing the decision-making. These points were then discussed, and decisions were made on important cross-team issues and coordination matters. Discord was mainly used as the medium of announcing major decisions, while the individual team members in the integration team were used as a medium to communicate points to their team. The integration team added value to the project by centralizing the responsibility and facilitating cross-team collaboration. The decisions made by the integration team could vary from coding practices to project planning, and cross-team coordination, and acted similar to a board that guided the project. They played a vital part in a project where communication was extremely important and helped a lot with ensuring cross-team integration.

11.3.2 Evaluation

Having one person from each team being part of a decision-making body is an efficient way of having all teams come to a collective decision about the project. This made communication with the integration team simple and clear, although it could potentially cause some problems if the representing team member did not convey points accurately. It also ended up being a way to facilitate conflict-solving between teams when disagreements in inter-team disciplines (such as PR and merges) got a bit heated. However, having one team decide everything related to the project as a whole necessitated frequent meetings and putting all the responsibility for those decisions on one member of each team. This often meant that one member of each team had to juggle an excessive number of tasks that were not directly related to the PBIs.

Reflection

While the Nexus model uses the integration team to facilitate inter-team dependent development, it is not meant as a catch-all for all project-wide related decisions. The nexus model does not address this, but some of the integration teams' added responsibility could be delegated to another team or teams structured in the same way as the integration team. Another important takeaway from the integration-team work was that the decisions and notes from the meeting were shared in a Google Drive folder. Even though it was pinned to the integration team Discord channel, many groups did not consult these meeting notes afterward. This resulted in confusion regarding this that was noted down in the integration team notes, but teams did not look at these notes.

11.4 Communication and challenges

After the last sprint review, each team member had to mix with a different team and construct a list of the most valuable aspects and lessons learned during this project. These lists were made by discussing a topic, with members of other teams. A recurrent topic on all of the lists was communication and coordination. These problems were specifically related to the following aspects:

- How is effective communication achieved, and how do we make sure that information is received and understood by everyone?
- How do we maintain transparency on decisions, progress, and impediments between six teams?
- How do we handle disagreements, conflicts, and impediments and make decisions in a flat hierarchy?

This revealed, that throughout all sprints, communication and coordination were a huge part of the project and was arguably the biggest problem as well. The different tools and methods used for communication will be discussed in this section.

11.4.1 Task Board

A task board was created to increase transparency and workflow between the developer team and the product manager. The task board is a digital board that displays the current status of any PBIs in the current sprint. This helped all teams visualize their workflows and PBIs. Furthermore, it increased the collaborative process across groups [42]. For instance, different teams or individuals were responsible for the implementation of a user in the database, and they could collaborate with the frontend team designing the related user UI [18]. A task board also shows the signs of bottlenecks in different tasks, if a PBI is stuck for a longer duration, you could identify it, address the issue promptly, and ensure implementation. The task board utilized in this project was Miro¹.

11.4.2 Integration team issues

As stated before, one of the ways the different teams used to communicate with each other was through the integration team. Having an integration team helped with transparency by gaining a better overview of the different group's progress. It also contributed to better cross-functionality between groups by being a channel for communication between groups and solving dependency issues. However, at times, the integration team was more focused on the general overview and progress, than actual integration between teams. Especially, the web and mobile teams lacked communication with each other. This caused some problems with teams working on some code that other teams were dependent on, or by spending time structuring data that may cause problems for other teams. This caused multiple setbacks throughout the project. If one were to utilize the full potential of the integration team, one could avoid these problems caused by a lack of communication between groups.

¹<https://miro.com/da/>

A lot of the decisions made by the integration team were also communicated to the rest of the teams by their representative instead of written announcements to everyone. This caused a few problems with the decisions being communicated differently depending on the representative and sometimes getting forgotten without a written record accessible for everyone to easily see. In general, having a better structure for how to document the agreements and decisions discussed in the integration team would be beneficial.

11.5 Product owner

The PO is a domain expert, and their responsibility is to formulate user needs as product backlog items, refine these items, and ensure that the value created goes towards the overarching goal of the project. Their approval of any item is pivotal to consider an item done.

Throughout the project, the PO often had to remind the Nexus team of the sprint goals, which helped keep the bigger picture in mind. During sprint reviews, the PO was clear and concise about the expectations set for the team, and although they had their clear priorities, a lot of freedom was given to the developers in discussing the value, size, design, and clarification of acceptance criteria. This was also needed, since, as a domain expert, the PO is not necessarily aware of the technical difficulties of, e.g., implementing a Google Login or resetting a password via email.

The product backlog was always kept up to date, and PBIs were selected based on how much they contributed to the actual sprint goal and how much value they added to the product. The selection took place in correspondence with the developers for a good mutual understanding. The acceptance criteria were defined by the developers, and at times these were accepted by the PO without further discussion. Since there was no discussion, it could at times, lead to the developers and the PO having different interpretations of the criteria.

In general, the PO was always open for meetings and easy to contact through Discord. She had frequent checkups with the groups to keep updated on their process, as well as realign expectations and priorities based on progress. At the end of Sprint 4, the PO moved back to Brazil, which led to communication being less frequent. However, to compensate, she then started to participate more in integration meetings, where she consulted with group representatives about progress, expectations, and priorities.

In the first couple of sprints, there were a lot of dependency problems between teams. Therefore, the PO took responsibility and started working on dependency management during sprint planning. In addition, she often reminded teams to deliver increments and thereby adhere to Scrum principles. Overall, the consensus surrounding the work with the PO is that it was satisfactory and enhanced our work and scrum process.

12 Discussion

In this section, the group will discuss their experience and use of different methods throughout this project. Firstly the group will discuss their experience with the chosen Nexus framework in section 12.1. Then the Management of the groups or teams in this project will be discussed in section 12.2, both with a focus on the internal communication within the group and also the external communication between all the groups. Then the group will discuss the overall management of the project in section 12.3 where a discussion regarding the groups experience with the project will take place. This includes collaboration with the product owner and then the stakeholders. The group will also discuss the technical aspects of the project in the Technical Debt section 12.4 and the Testing section 12.5. The discussion section will end with discussing the further development section 12.6.

12.1 Nexus framework

As mentioned prior in this report, the agile scaling method the groups decided together to use for this project was the Nexus framework.

The group has found the Nexus framework to have been very beneficial for communication and collaboration across all the teams with the improvement of the usage of the integration teams throughout the sprints. At the start of the project, one group member was assigned to be a part of the integration team, however, when looking back now, it might have been an idea to switch the member responsible for the integration team each sprint. Even though the integration team concept greatly improved the project overall, it turned out to be a lot of hard work and time-consuming for the member to be a part of the integration team. After the integration meetings, the group member would return to the group and share what was discussed at the meeting, however, it might have been more insightful for all the group members to have switched each sprint so the hard work regarding the integration meeting would have been more evenly spread and not placed as "extra" work the whole project on one member.

12.2 Management of the groups

Internal communication

Throughout the work and implementation of PBI's the group has been working with pair-programming. This is where the group split up into pairs and each pair works on a different PBI. This method was chosen due to the groups prior experience with it and has proven to be very efficient and effective for the group in this project as well. Each pair gained more knowledge of the code they were working with during implementation, and then when the pairs got new PBIs they shared the knowledge on what they have learned and implemented. Each sprint the members would switch who they paired with to create the best learning environment and learn from each other.

External communication

Due to this project being the group's first "cross-team" project, it was quite hard for the group to get used to in the beginning. There arose a lot of issues and conflicts in the beginning of the project with the group and the other teams in the project. However, during the second sprint, the integration team took their work a step up, and the communication between the groups improved. though with a few minor exceptions

12.3 Management of the project

When the group first started with this project, they took over the work done on this Educado project by the previous semester's groups. The project the group was given only had the bare minimum of functionality, only allowing the user to see the UI of creating a course and a default profile page. As the code received was very unstructured, unorganized, and hugely lacked documentation, it took the group a great amount of time to become familiarized with the code and produce good and efficient code. The state of the code was one of the bigger weaknesses of the project. Throughout the project, the group worked with the existing code but made adaptations when needed and used code extension tools to help make sure there was documentation in their code.

Another weakness of the project that the group had to adapt and work around was the lack of control when the group disagreed with certain decisions previously made. An example of this is the type of database that is being used for the Educado project. The database used is a document database, which the group suggested that it was changed to a relational database. This was suggested because the code in the project was already structured and handled the database as a relational database. However, in a document database, there can be multiple documents in the same collection, and they can have any shape and do not have to follow a certain structure. If all the

documents do not follow a certain structure it can strongly disrupt the code, which is written only to follow a specific style of document. This would not have been an issue if the database type was changed, but it was not in the hands of the group to decide and therefore had to adapt to the decisions.

Collaboration with the Product Owner

Throughout the project, the group had regular contact with the product owner. For the first four sprints, the PO was present in Denmark with the development teams. For the 5th and 6th sprints, she returned to Brasilia and therefore was only available online. During the first sprints, every group had to learn who the PO was and discuss expectations from each other. The groups experience with the PO was very positive, some miscommunication happened throughout the project but it was handled well between the group and the PO. It can be concluded that communication was easier with the PO when the PO was physically present rather than online. However, the online communication worked well until the end of the project.

Collaboration with the Stakeholders

A part of agile development is the collaboration and communication between the development teams and the stakeholders. In this project, the PO acted as a middleman between the development teams and the stakeholders, however, due to time difference and a language barrier the development teams had minimal contact with the stakeholders in Brazil. There was 1 session with a user test where a Brazilian stakeholder tested the groups work, the test went well but that was the only contact that the group had with the stakeholders. It can be discussed that the PO is a stakeholder as well and also represents the other stakeholders well enough that further contact was not needed. However, the PO was not the target of the product to the same extent that the stakeholder was, and some feedback might have been lost in the process.

Pull requests

A problem that the group encountered during the project was the sizes of the pull requests that were made to the different branches. The group had had some recurring issues with creating very large pull requests. It is generally not good for the project when larger pull requests are committed rather than many smaller pull requests. The smaller pull requests are usually better because the changes and the code that had to be reviewed are smaller and take a shorter time to look at in comparison to larger pieces of code with big changes. The group aimed at creating smaller pull requests toward the end of the 5th and 6th sprints. As mentioned in the section *Sprint 6 Review 9.7.5*, the group managed to create more pull requests than they did at the beginning of the project. However, these pull requests were still on the larger end regarding size and changes. The group should have focused more on the sizes of their pull requests earlier in the project before it became a problem. However, they still managed to create successful pull requests throughout this sprint.

12.4 Technical debt

Technical debt is a software engineering concept where the developers undertake a path that results in a developed solution of substandard quality. Early on in the project, the group was introduced to this concept. Although the group did not apply this concept in the beginning, as time went on, the group got a better feel of agile development and grasped an understanding of how technical debt can be applied. A concept of agile development is to produce value for the stakeholders and to produce more value, the group applied technical debt in the following manners.

In the early stages of the project, Jest testing was used for the frontend, though this testing method was too time-consuming, therefore inefficient, and was quickly deemed obsolete. Furthermore, it was discussed that front-end testing should be briefly suspended until a more suitable testing method was discovered. After the discovery and implementation of Cypress testing, the technical debt was re-paid.

The group also used to write "perfect" code but discovered that meeting the sprint goal was difficult and, in some cases, impossible. The group chose to lower the code quality by choosing sub-optimal solutions if it meant pushing features more frequently and thus delivering more value. An example of this would be a part of the code for the PBI regarding the step-by-step checklist and can be seen in *Appendix A*. The design of the checklist is hard coded in a long condition. This was done due to the group not having much experience designing frontend with Tailwind. The group discussed that this code could be rewritten by making use of functions or arrays, but opted out of re-factoring the code since it would not bring more value, and thus moved on to developing other features.

12.5 Testing with Cypress vs other testing frameworks

As previously mentioned the group used a tool for testing called Cypress. Cypress is an intuitive syntax and built-in test runner. It enabled us to have an easy and reliable way to test the web components in our browser. The group did look into some other testing frameworks. The main ones we looked at were Cucumber, TestCafe, and Playwright. Here are listed some strengths and considerations that were found for the different ones. Unlike other testing frameworks, Cypress is built for JavaScript.

Cypress

Syntax: Generally speaking the learning curve was found to be intuitive and easy to understand in terms of the syntax for the framework.

Real-Time Interaction: Cypress has the ability to use real-time interaction with the browser during tests, which can provide dynamic and interactive testing environments.

General Consideration: The only real consideration is that it is tailored for JavaScript but that is what we are writing in so it fits in this project.

Cucumber

Natural Language: Cucumber allows for the natural language of the test to feature a more easily understandable scenario. Enabling more interaction of tests with people of lower technical levels.

General Consideration: The nature of Cucumber and generally BDD syntax-based frameworks like these, make for a difficult learning curve for those new to it.

TestCafe

Multi-Browser support: TestCafe supports browser testing which also works across different browser types.

No Plugins: Requires no form of plugins for browsers simplifying the process a lot.

General Considerations: Unlike something like Cypress, TestCafe might have some limitations regarding real-time interaction during a test.

Playwright

Multi-Browser support: TestCafe supports browser testing which also works across different browser types.

Multi-Platform Automation: Playwright offers an automation framework for both web, mobile, and desktop applications.

General Considerations: The learning curve of Playwright's extensive function may introduce a too steep learning curve for the teams' experience compared to simpler testing frameworks.

The testing framework that was used in the end was Cypress. The reason for this is that it was good for simple tests and had real-time browser interaction. Furthermore, it is a framework made for web testing and is specialized for this form of development the group is engaging in.

12.6 Future development

A part of the discussion is to look into further development of the project, even though the Educado project is big and there is plenty more to implement. However, due to limited time on the project, there were things the group did not have time to work on, and they should be the starting point for future developers. The first important thing in the project's time frame that should be prioritized is the PBIs from the last sprint that did not have all their acceptance criteria accomplished. Secondly, the foundation of the content creator platform should be developed since the product owner highly values it. And lastly, the original task that the group was assigned and was supposed to develop towards the end of the project was a Virtual Tutor micro-service.

A more in-depth discussion will be included on some handpicked features.

Upload image and video files

As mentioned throughout the sprints, the group started with a PBI that included buckets to handle file uploads to the Google Cloud Platform (GCP). This feature could first be implemented after one of the other groups developed a dependency on bucket communication. The group communicated with the other group to get an understanding of the bucket implementation and developed a functioning image and video upload on the web. However, this feature

had a short lifespan since the other group working with the buckets changed the structure of communication with the GCP and did not communicate or discuss this change, leading to the feature not functioning. Unfortunately, this was found at the end of the project, and the group could only apply small fixes to stop the website from crashing, as previously explained in implementation 10.2.2. This is a vital feature not only for the content creators but for the concept of the Educado platform and must be fixed. There are multiple ways to implement a fix on this feature, the most optimal being a collaboration between the groups to ensure an efficient solution.

Lectures and exercises The group implemented the design and most of the functionality of the lectures and exercises during the third and fourth sprints. The user test held in the fourth sprint revealed that a redesign was needed. The stakeholders who were the participants of the user test wanted the lectures and exercises to be re-modeled, rather than having them as two separate elements, they should be options under a single component element. The only redesign done for the lecture and exercises was on the list displaying them, which was completed in sprint 6. The functionality for choosing between a video or text lecture was implemented, though as described above in the part on "Upload image and video files", the buckets communication was changed, and therefore a video lecture cannot contain a video. Textual lecture can be created, but without a comprehensive textual editing experience, which was the only remaining PBI not developed at all in sprint 6. Most of the functionality for lectures and exercises are implemented, except for the two that were just described and should be considered for further development as well as the exercise and video create/edit pop-ups need a complete visual overhaul to match the new style guide.

13 Conclusion

Throughout this project, the group has worked on the web application part of the Educado project. The web application is the interface that allows content creators to create courses with sections, lectures, and exercises for the users of the mobile application.

The project contained 6 sprints each with its own sprint goal, the group has somewhat fulfilled these sprint goals, even though the group encountered many issues with the code and the communication between the different groups. This has been discussed further in each sprint section of this report.

The goals for this project was to have multiple groups work on the same project and all the groups' work was successfully integrated into a single project, this project being the Educado platform product. Despite miscommunication and experience gained during this project, all the groups, including this group, have successfully implemented their own work into the project and merged it with the work of all the other groups.

The group was assigned their own focus area of implementing a virtual tutor, which was their goal to work on, however, this goal changed due to prioritization by the Product Owner of implementing and stabilizing functionalities on course creation rather than adding new features. The group agreed with the Product Owner to focus only on the course creation rather than working on the virtual tutor. At the end of the project, the group completed their work on this focus area by working on the functionality and design of the course creation.

Within all these goals for the project and the group as based on their work written in this report, the group would conclude their work on the project as successful therefore they have fulfilled the goals of the Educado project.

References

- [1] Aalborg University, *COMPLEX BACK-END SOFTWARE 2022/2023*, en. [Online]. Available: <https://moduler.aau.dk/course/2022-2023/DSNSWCB521> (visited on 12/05/2023).
- [2] United Nations, *FN Brasil*, no, Sep. 2023. [Online]. Available: <https://www.fn.no/Land/brasil> (visited on 12/05/2023).
- [3] World Bank Group, *World Bank Poverty Brazil*, en, Apr. 2023. [Online]. Available: https://databankfiles.worldbank.org/public/ddpext_download/poverty/987B9C90-CB9F-4D93-AE8C-750588BF00QA/current/Global_POVEQ_BRA.pdf (visited on 12/05/2023).
- [4] C. Belandi, “Poverty Record Brazil,” en, *IBGE*, Dec. 2022. [Online]. Available: <https://agenciadenoticias.ibge.gov.br/en/agencia-news/2184-news-agency/news/35695-poverty-hits-a-record-in-2021-62-5-million-persons-highest-level-since-2012> (visited on 12/05/2023).
- [5] L. Rodrigues, “IBGE inequality,” br, *IBGE*, Jun. 2023. [Online]. Available: <https://agenciabrasil.ebc.com.br/educacao/noticia/2023-06/ibge-revela-desigualdade-no-acesso-educacao-e-queda-no-analfabetismo> (visited on 12/05/2023).
- [6] B. Lima and L. Guilherme Gerbelli, “G1 Brazil higher education,” br, *G1*, Aug. 2020. [Online]. Available: <https://g1.globo.com/economia/concursos-e-emprego/noticia/2020/08/11/no-brasil-40percent-dos-jovens-com-ensino-superior-nao-tem-emprego-qualificado.ghtml>.
- [7] I. Paiva, “CNN Brazil Unemployment,” br, *CNN Brasil*, Aug. 2023. [Online]. Available: <https://www.cnnbrasil.com.br/economia/taxa-de-desemprego-fica-em-79-no-trimestre-encerrado-em-julho-diz-ibge/> (visited on 12/05/2023).
- [8] L. Cardoso Queiroz Melo, *Educado project introduction*, English, Aalborg University in Copenhagen, Sep. 2023. [Online]. Available: https://www.moodle.aau.dk/pluginfile.php/3137156/mod_resource/content/1/Software%205%20-%20Project%20Introduction%20%281%29.pdf (visited on 06/12/2023).
- [9] D. Britze and Et al., *Mobile Education Project Educado*, en, 2021. [Online]. Available: https://vbn.aau.dk/ws/portalfiles/portal/468944083/PAEE_ALE_2021_103.pdf (visited on 12/05/2023).
- [10] European Comission, *What is Erasmus+?* [Online]. Available: <https://erasmus-plus.ec.europa.eu/about-erasmus/what-is-erasmus> (visited on 12/06/2023).
- [11] United Nations, *The 17 goals*. [Online]. Available: <https://sdgs.un.org/goals> (visited on 12/06/2023).
- [12] Anton Olsen, Charlotte Thaarup, Johan Nissen Riedel, et al., *Educado An Educational Mobile Application*, English, Dec. 2022. [Online]. Available: <https://projekter.aau.dk/projekter/files/510008807/EducadoAnEducationalMobileApplication.pdf> (visited on 12/08/2023).
- [13] R. S. Pressman and B. R. Maxim, *Software engineering: a practitioner’s approach*, eng, Ninth edition. New York, NY: McGraw-Hill Education, 2020, ISBN: 978-1-260-54800-6.
- [14] Atlassian, *Agile Scrum Roles*, en, Dec. 2023. [Online]. Available: <https://www.atlassian.com/agile/scrum/roles> (visited on 12/05/2023).
- [15] M. D. Kadenic, K. Koumaditis, and L. Junker-Jensen, “Mastering scrum with a focus on team maturity and key components of scrum,” *Information and Software Technology*, vol. 153, p. 107079, Jan. 2023, ISSN: 0950-5849. DOI: 10.1016/j.infsof.2022.107079. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584922001884> (visited on 12/05/2023).
- [16] H. F. Cervone, “Understanding agile project management methods using Scrum,” *OCLC Systems & Services: International digital library perspectives*, vol. 27, no. 1, pp. 18–22, Jan. 2011, Publisher: Emerald Group Publishing Limited, ISSN: 1065-075X. DOI: 10.1108/10650751111106528. [Online]. Available: <https://doi.org/10.1108/10650751111106528> (visited on 12/05/2023).
- [17] K. Schwaber and J. Sutherland, *The Scrum Guide™*, en-us, 2017. (visited on 12/06/2023).
- [18] *Scrum Development Team - Who’s in it? | Roles & Responsibilities*, en. [Online]. Available: <https://www.knowledgehut.com/tutorials/scrum-tutorial/development-team> (visited on 12/05/2023).
- [19] S. Salimi, *Development Team*, en. [Online]. Available: <https://www.agile-academy.com/en/agile-dictionary/development-team/> (visited on 12/05/2023).

- [20] V. Gulliksen Stray, N. B. Moe, and T. Dingsøyr, “Challenges to Teamwork: A Multiple Case Study of Two Agile Teams,” en, in *Agile Processes in Software Engineering and Extreme Programming*, A. Sillitti, O. Hazzan, E. Bache, and X. Albaladejo, Eds., ser. Lecture Notes in Business Information Processing, Berlin, Heidelberg: Springer, 2011, pp. 146–161, ISBN: 978-3-642-20677-1. DOI: 10.1007/978-3-642-20677-1_11.
- [21] K. Schwaber and J. Sutherland, “The Scrum Guide,” Eng, vol. Ken Schwaber & Jeff Sutherland, p. 13, Nov. 2020. [Online]. Available: https://www.moodle.aau.dk/pluginfile.php/2895744/mod_resource/content/1/2020-Scrum-Guide-US.pdf.
- [22] scrum.org, *Scaling Scrum with Nexus*, en. [Online]. Available: <https://www.scrum.org/resources/scaling-scrum> (visited on 09/22/2023).
- [23] Zoho.com, *What is a sprint review? Definition, purpose, best practices / Zoho Sprints*, en. [Online]. Available: <https://www.zoho.com/sprints/sprint-reviews.html> (visited on 12/06/2023).
- [24] Scrum.org, *What is a Sprint Review? / Scrum.org*, en. [Online]. Available: <https://www.scrum.org/resources/what-is-a-sprint-review> (visited on 12/06/2023).
- [25] K. Bittner, P. Kong, E. Naiburg, and D. West, *The Nexus Framework for Scaling Scrum: Continuously Delivering an Integrated Product with Multiple Scrum Teams*, en. Addison-Wesley Professional, Dec. 2017, Google-Books-ID: BUhBDwAAQBAJ, ISBN: 9780134682716.
- [26] P. Kong and S. Bourk, *An Introduction to the Nexus™ Framework*, en, Jun. 2016. [Online]. Available: https://scrumorg-website-prod.s3.amazonaws.com/drupal/2016-06/An%20Introduction%20to%20the%20Nexus%20Framework%20-%20June%202016_0.pdf (visited on 12/06/2023).
- [27] Scrum.org, *Online Nexus Guide / Scrum.org*, en, Jan. 2021. [Online]. Available: <https://www.scrum.org/resources/online-nexus-guide> (visited on 12/06/2023).
- [28] A. Karabulut, “Understanding Clean Architecture and Domain-Driven Design (DDD),” May 2023. [Online]. Available: <https://medium.com/bimar-teknoloji/understanding-clean-architecture-and-domain-driven-design-ddd-24e89caabc40>.
- [29] A. Afrane-Okese, “What is Clean Architecture in software development?,” Mar. 2022. [Online]. Available: <https://medium.com/dvt-engineering/what-is-clean-architecture-in-software-development-89a7f5639f07>.
- [30] J. Machen, “MICROSERVICE ARCHITECTURE : BENEFITS AND CHALLENGES,” *SmartWave*, [Online]. Available: <https://www.smartwavesa.com/blog-articles/microservice-architecture-benefits-and-challenges/>.
- [31] J. Nemer, *Advantages and Disadvantages of Microservices Architecture*, Jul. 2023. [Online]. Available: <https://cloudacademy.com/blog/microservices-architecture-challenge-advantage-drawback/>.
- [32] Luiza Cardoso Queiroz Melo, *Educado S5*, en, Dec. 2023. [Online]. Available: <https://miro.com/app/board/uXjVPSIRydA=> (visited on 09/22/2023).
- [33] Jest, *Jest*, en, Dec. 2023. [Online]. Available: <https://jestjs.io/> (visited on 12/20/2023).
- [34] Testim, *Jest Testing: A Helpful Introductory Tutorial - Testim.io*, en-US, Mar. 2022. [Online]. Available: <https://www.testim.io/blog/jest-testing-a-helpful-introductory-tutorial/> (visited on 12/20/2023).
- [35] Jest, *Snapshot Testing · Jest*, en, Sep. 2023. [Online]. Available: <https://jestjs.io/docs/snapshot-testing> (visited on 12/20/2023).
- [36] Jest, *Testing Asynchronous Code · Jest*, en, Sep. 2023. [Online]. Available: <https://jestjs.io/docs/asynchronous> (visited on 12/20/2023).
- [37] Testim, *Cypress vs. Selenium: What’s the Better Testing Framework?* en-US, Aug. 2022. [Online]. Available: <https://www.testim.io/blog/cypress-vs-selenium/> (visited on 12/16/2023).
- [38] Cypress, *Introduction to Cypress / Cypress Documentation*, en, Aug. 2023. [Online]. Available: <https://docs.cypress.io/guides/core-concepts/introduction-to-cypress> (visited on 12/16/2023).
- [39] Refactoring Guru, *Observer*, en, Dec. 2023. [Online]. Available: <https://refactoring.guru/design-patterns/observer> (visited on 12/20/2023).
- [40] Educado, *Figma*, en, Section: Design. [Online]. Available: <https://www.figma.com/file/BCEVQYvMDrcBE9qqehr9HL/Mobile-Education-UI> (visited on 12/16/2023).

- [41] Microsoft, *Microsoft Azure REST API Guidelines*, Dec. 2023. [Online]. Available: <https://github.com/microsoft/api-guidelines/blob/vNext/azure/Guidelines.md#microsoft-azure-rest-api-guidelines>.
- [42] B. van Ruler, “Agile public relations planning: The Reflective Communication Scrum,” *Public Relations Review*, Digital Publics, vol. 41, no. 2, pp. 187–194, Jun. 2015, ISSN: 0363-8111. DOI: 10.1016/j.pubrev.2014.11.008. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0363811114001805> (visited on 12/06/2023).

Appendix A - Checkbox Implementation

```

1  { /* Check-box list */}
2  <div className="border-y py-8 w-5/6 border-grayMedium flex flex-col space-y-4 ">
3
4      { /* General Information */}
5      {tickChange === 0 ?
6          <label htmlFor="check1" className="pl-2 flex items-center border-l-4 border-primary" >
7              <input className="mr-2 text-primary rounded" type="checkbox" id="check1" disabled />
8              Informações gerais</label>
9          :
10         <label htmlFor="check1" className="pl-3 flex items-center" >
11             <input className="mr-2 text-primary rounded" type="checkbox" id="check1" disabled checked/>
12             Informações gerais</label>
13     }
14
15
16     { /* Course sections */}
17     {tickChange === 0 ?
18         <label htmlFor="check2" className="pl-3 flex items-center" >
19             <input className="mr-2 text-primary rounded" type="checkbox" id="check2" disabled/>
20             Seções do curso</label>
21         :
22         tickChange === 1 ?
23         <label htmlFor="check2" className="pl-2 flex items-center border-l-4 border-primary" >
24             <input className="mr-2 text-primary rounded" type="checkbox" id="check2" disabled/>
25             Seções do curso</label>
26         :
27         <label htmlFor="check2" className="pl-3 flex items-center" >
28             <input className="mr-2 text-primary rounded" type="checkbox" id="check2" disabled checked/>
29             Seções do curso</label>
30     }
31
32
33     { /* Review course */}
34     {tickChange < 2 ?
35         <label htmlFor="check3" className="pl-3 flex items-center" >
36             <input className="mr-2 text-primary rounded" type="checkbox" id="check3" disabled/>
37             Revisar curso</label>
38         :
39         <label htmlFor="check3" className="pl-2 flex items-center border-l-4 border-primary" >
40             <input className="mr-2 text-primary rounded" type="checkbox" id="check3" disabled/>
41             Revisar curso</label>
42     }
43
44 </div>

```

Listing 15: Partial code of checkbox implementation.